

A Perturbation-Augmented Unsupervised Learning Framework for Integer Linear Programming

Yufan Deng

Laboratory for Big Data and Decision,
College of Systems Engineering,
National University of Defense
Technology
Changsha, China
dengyufan24@nudt.edu.cn

Tianle Pu

Laboratory for Big Data and Decision,
College of Systems Engineering,
National University of Defense
Technology
Changsha, China
putl22@nudt.edu.cn

Zhijing Hu

Laboratory for Big Data and Decision,
College of Systems Engineering,
National University of Defense
Technology
Changsha, China
huzhijing@nudt.edu.cn

Zijie Geng*

University of Science and Technology
of China
Hefei, China
zijiegeng@mail.ustc.edu.cn

Li Zeng

Laboratory for Big Data and Decision,
College of Systems Engineering,
National University of Defense
Technology
Changsha, China
zengli24@nudt.edu.cn

Xingchen Hu

Laboratory for Big Data and Decision,
College of Systems Engineering,
National University of Defense
Technology
Changsha, China
xhu4@ualberta.ca

Kuihua Huang

Laboratory for Big Data and Decision,
College of Systems Engineering,
National University of Defense
Technology
Changsha, China
kuihh@163.com

Junjie Wu

School of Economics and
Management, Beihang University
Beijing, China
MIIT Key Laboratory of Data and
Decision Intelligence, Beihang
University
Beijing, China
wujj@buaa.edu.cn

Changjun Fan*

Laboratory for Big Data and Decision,
College of Systems Engineering,
National University of Defense
Technology
Changsha, China
fanchangjun@nudt.edu.cn

Abstract

Integer linear programming (ILP) plays a pivotal role in combinatorial optimization, with applications ranging from logistics to scheduling. Recently, unsupervised learning has emerged as a promising paradigm for solving ILPs without costly labeled solutions. However, existing methods often converge prematurely to polarized probability distributions, losing solution diversity and trapping the process in local optima. To alleviate this, we propose PUMA, a Perturbation-augmented Unsupervised learning fraMework for integer linear programming. PUMA introduces a stochastic neighborhood perturbation (SNP) operator that injects structured noise into the sampled solution space. The perturbation magnitude follows an exponential decay schedule, promoting exploration in early stages and refinement later. Theoretically, we show that PUMA optimizes a penalized objective with a dynamically decaying regularization term. Extensive experiments on benchmark ILP problems demonstrate that PUMA significantly mitigates the early collapse of solution

diversity. It reduces the primal gap by 39.73% on average compared to state-of-the-art baselines. Furthermore, when used as a heuristic to warm-start traditional solvers, PUMA enables Gurobi to achieve an additional 91.17% reduction in the primal gap, and SCIP a 58.12% reduction.

CCS Concepts

• **Computing methodologies** → **Learning in probabilistic graphical models; Machine learning; Machine learning approaches.**

Keywords

Learning to Optimize, Machine Learning for Combinatorial Optimization, Integer Linear Programming

ACM Reference Format:

Yufan Deng, Tianle Pu, Zhijing Hu, Zijie Geng, Li Zeng, Xingchen Hu, Kuihua Huang, Junjie Wu, and Changjun Fan. 2026. A Perturbation-Augmented Unsupervised Learning Framework for Integer Linear Programming. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD 2026)*, August 9–13, 2026, Jeju Island, Republic of Korea. ACM, New York, NY, USA, 23 pages. <https://doi.org/10.1145/3770855.3817737>

Resource Availability:

The source code is publicly available at <https://doi.org/10.5281/zenodo.20508154> and at <https://github.com/Joky02/PUMA>.

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *KDD 2026, Jeju Island, Republic of Korea.*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2259-2/2026/08
<https://doi.org/10.1145/3770855.3817737>

1 Introduction

Integer linear programming (ILP) plays a central role in diverse practical applications, spanning production planning [5, 38], supply chain scheduling [6, 15], and resource allocation [23, 30]. Due to its high modeling efficiency, ILP has attracted significant attention, leading to the development of numerous effective algorithms over the past decades, such as branch-and-bound [25] and branch-and-cut [13]. However, as a well-known NP-hard problem, solving ILPs at scale remains computationally challenging, limiting the practical applicability of these traditional methods to large industrial instances.

In recent years, machine learning (ML) has emerged as a promising alternative, leveraging data-driven patterns to accelerate optimization. Existing ML approaches largely fall into two categories. First is learning to guide traditional solvers (e.g., branching policies [9]), second is training neural networks to directly predict high-quality solutions [28]. In this work, we focus on the latter paradigm. Although such methods show potential in speeding up ILP solving, they typically rely on supervised learning from labeled solutions, which are expensive or often impractical to obtain in real settings. Unsupervised learning offers an appealing alternative by eliminating the need for solution labels altogether. Among these, DiffILO [11] first transforms the ILP problems into continuous ones and then optimizes them toward the optimal objective direction. Specifically, it formulates ILP solving as a probabilistic expectation optimization, jointly optimizing the objective function while progressively reducing constraint violations, by sampling solutions from the probabilistic space.

Despite this progress, a fundamental limitation persists: current unsupervised methods tend to converge prematurely to polarized probability distributions (near 0 or 1), which drastically reduces the diversity of sampled solutions. As illustrated in Figure 1, this loss of diversity corresponds to a rapid decline in exploration capability, ultimately trapping the optimization in suboptimal local minima and preventing the discovery of better solutions.

To overcome this limitation, we propose **PUMA** (**P**erturbation-augmented **U**nsupervised learning fra**M**ework for integer linear progr**A**mming). PUMA explicitly maintains solution diversity during early learning stages through a novel stochastic neighborhood perturbation (SNP) operator, which injects structured noise into sampled solutions within a Hamming-distance-defined neighborhood. The perturbation magnitude follows an exponentially decaying schedule, encouraging broad exploration initially and shifting gradually toward focused refinement. Theoretically, we demonstrate that PUMA effectively optimizes a penalized objective with a dynamic regularization term that diminishes over time, guaranteeing convergence to high-quality solutions.

The contributions of this work are summarized:

- (i) We propose PUMA, a novel unsupervised framework that incorporates a stochastic neighborhood perturbation (SNP) operator with an exponentially decaying schedule to inject structured noise, thereby promoting solution diversity and preventing entrapment in suboptimal local minima.
- (ii) We establish a solid theoretical foundation demonstrating that the SNP operator, under the exponentially decaying

schedule, functions as a dynamic landscape-smoothing regularizer whose magnitude decays linearly with the perturbation rate. This theoretically validates its efficacy in balancing exploration and refinement.

- (iii) Extensive experiments on challenging ILP benchmarks demonstrate the superiority of PUMA. Results show that PUMA outperforms competitors with an average reduction of 39.73% in the primal gap. Furthermore, it significantly accelerates commercial solvers, reducing the primal gaps of Gurobi and SCIP by up to 91.17% and 58.12%, respectively, while exhibiting robust generalization to large-scale instances and MIPLIB problems.

The rest of this paper is organized as follows. Section 2 reviews the related work. Section 3 provides the necessary background. Section 4 formulates the PUMA framework and its theoretical properties. Section 5 presents extensive experiments on synthetic and real-world benchmarks. Finally, Section 6 concludes the paper and discusses future directions. Additional implementation details, extended results, and complete proofs are provided in the Appendix.

2 Related Work

2.1 ML-based Component in B&B Solver

One line of research integrates machine learning (ML) components into classical exact algorithms, such as the Branch-and-Bound (B&B) method for solving Integer Linear Programming (ILP) problems. The efficiency of the B&B algorithm heavily depends on its internal heuristics, particularly for node and branching variable selection. This complex, data-rich decision-making environment, often coupled with a limited theoretical understanding of a specific problem’s structure, makes B&B a fertile ground for ML techniques.

A significant body of work focuses on applying ML to enhance decision-making within the B&B framework [1, 22]. Much recent research has explored the “learning to branch” paradigm. This includes learning policies for branching variable selection [9, 14, 22, 24, 39, 44, 49], node selection [17, 39, 47], and cutting plane selection [20, 26, 29, 35, 41]. Among these, learning to select branching variables remains a primary focus, as this choice critically impacts the size of the search tree and, consequently, the overall efficiency of the B&B algorithm.

2.2 ML-Based Solution Prediction

Another line of research leverages ML to predict ILP solutions directly [4, 7, 11, 22, 48, 50]. A pioneering work in this area, Neural Diving [28], frames the ILP as a supervised learning task on a bipartite graph to predict integer variable assignments. Specifically, ND predicts a partial solution and utilizes a SelectiveNet [10] to decide which variables to fix.

Building on this foundation, a prominent trend is the development of hybrid ML-solver paradigms, where an ML-predicted solution is used to warm-start or guide a traditional solver’s search process [16, 19, 27, 48]. To improve the quality of the final solution, these methods often incorporate sophisticated search mechanisms, such as trust-region search [16, 19, 27, 31–34] or large neighborhood search (LNS) enhanced with advanced optimization techniques [18, 36, 40, 45, 46].

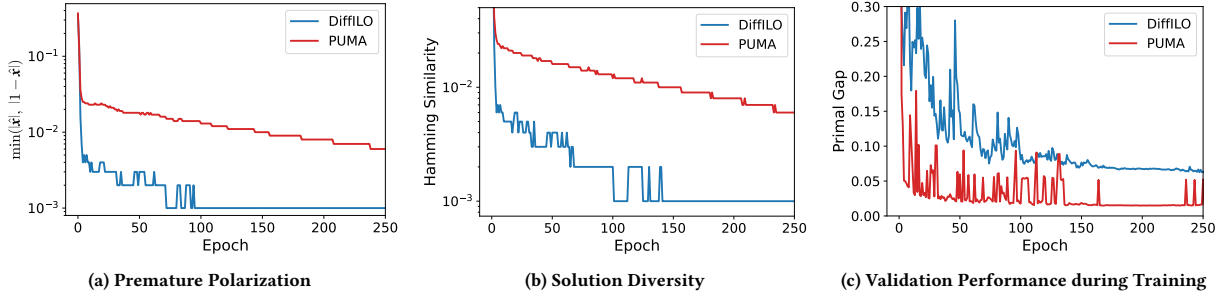


Figure 1: Analysis of PUMA’s training dynamics on the Maximum Independent Set (IS) problem. This figure illustrates the core motivation for our work: rapid probability polarization in existing unsupervised methods (e.g., DiffiLO [11]) leads to rapid diversity decline and subsequent learning stagnation. (a) **Premature Polarization:** This subfigure compared the polarization of probabilistic output distribution during training, measured by the metric $\min\{|\hat{x}|, |1 - \hat{x}|\}$. (b) **Solution Diversity:** This subfigure compares the diversity of solutions sampled from DiffiLO and PUMA, measured by the average Hamming distance between solution pairs. (c) **Training Performance:** This subfigure shows the primal gap of the best solution found on a validation set relative to the Best Known Solution (BKS) as training progresses.

2.3 Unsupervised Learning for Combinatorial Optimization

Unsupervised learning has emerged as a critical paradigm for Combinatorial Optimization (CO), offering a label-free framework that circumvents the need for large-scale, high-quality labeled datasets. The *Erdős Goes Neural* framework [21] introduced a seminal unsupervised approach for CO, stimulating numerous follow-up studies [3, 36, 37, 43]. Among these, DIMES [36] learns a continuous Bernoulli-style parameterization with a GNN backbone for graph CO problems, an approach close in spirit to the probabilistic + GNN formulation we adopt. A primary challenge in this domain, however, is the necessity of designing problem-specific loss functions, which limits applicability to general ILPs.

This challenge is particularly acute for ILP. Unlike more structured CO problems, the general formulation of ILP makes it difficult to design a universal, handcrafted surrogate. To address this, DiffiLO [11] proposes a general unsupervised framework based on probabilistic relaxation, avoiding problem-specific design. However, such methods suffer from a rapid decline in solution diversity that traps optimization in poor local minima, limiting effectiveness on complex instances. Compared to DIMES and DiffiLO, our contribution is orthogonal to the probabilistic reformulation and GNN parameterization: we introduce the stochastic neighborhood perturbation (SNP) operator with an exponentially decaying schedule, which directly counteracts premature polarization and significantly improves final solution quality.

3 Preliminaries

3.1 Integer Linear Programming

An integer linear programming (ILP) problem is a type of the combinatorial optimization problem. In these problems, all of the variables are constrained to take integer values. Formally, an integer linear programming problem M can be defined as

$$\min_{\mathbf{x}} \quad \mathbf{c}^\top \mathbf{x} \quad \text{s.t.} \quad \mathbf{A}\mathbf{x} \leq \mathbf{b}, \mathbf{x} \in \{0, 1\}^n, \quad (1)$$

where $\mathbf{x} = (x_1, \dots, x_n)^\top$ collects the n binary decision variables to be optimized. The vector $\mathbf{c} \in \mathbb{R}^n$ is the objective coefficient vector, and $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{b} \in \mathbb{R}^m$ specify the m linear constraints. A solution $\mathbf{x}$ is called feasible if and only if it satisfies all the constraints. In this paper, we focus on the binary form because, following the construction of [28], an integer linear program involving general integers can be reduced to the above form.

3.2 Graph Neural Networks for Bipartite ILP Graph Representations

A lossless bipartite graph representation of ILP was first proposed by Gasse et al. [9] as an input for neural embedding networks. In this representation, the n decision variables of an integer linear program form the set of right-side variable nodes, while the m linear constraints constitute the set of left-side constraint nodes. An edge connects a variable node i to a constraint node j if variable i is present in constraint j , with the edge weight corresponding to the variable’s coefficient in that constraint.

Building on this representation, Gasse et al. [9] also introduced a Graph Convolutional Network (GCN) architecture specifically designed for these bipartite graphs. The model takes the bipartite graph $\mathcal{G} = (\mathcal{C} \cup \mathcal{V}, \mathcal{E})$ as input, where $\mathcal{C}$ and $\mathcal{V}$ are the feature matrices for the constraint and variable nodes, respectively, and $\mathcal{E}$ is the set of edges with their features.

The core of this architecture is a single graph convolution operation, which is divided into two interleaved half-convolutions due to the bipartite structure. These can be viewed as two message-passing steps, formally described by the following equations:

$$\begin{aligned} c_i &\leftarrow f_C \left(c_i, \sum_{(i,j) \in E} g_C(c_i, v_j, e_{i,j}) \right), \quad \forall i \in \mathcal{C} \\ v_j &\leftarrow f_V \left(v_j, \sum_{(i,j) \in E} g_V(c_i, v_j, e_{i,j}) \right), \quad \forall j \in \mathcal{V} \end{aligned} \quad (2)$$

where f_C, f_V, g_C , and g_V are 2-layer perceptrons with ReLU activation functions.

Following this graph convolution layer, each node’s features have incorporated information from its one-hop neighbors. Finally, the model discards the constraint nodes and feeds the updated feature vectors of the variable nodes into another 2-layer perceptron. This is combined with a masked softmax activation function to produce a probability distribution over the candidate branching variables, which can directly guide the branching decision in a solver. A key advantage of this architecture is that it is well-defined for any input graph size and is permutation-invariant, making it highly suitable for handling the variable structures of ILP problems.

4 PUMA Framework

In this section, we present PUMA, a framework specifically engineered to mitigate the premature probability polarization and loss of solution diversity identified in Section 1. Central to our approach is the integration of a stochastic neighborhood perturbation (SNP) operator into the probabilistic ILP model. By employing an exponentially decaying perturbation schedule, PUMA shifts the emphasis from broad exploration at the outset to local refinement as learning progresses.

4.1 Framework Overview

As demonstrated in Figure 2, PUMA is designed to learn a robust solution-generating policy by integrating three key components: (i) GNN-based Parameterization to capture structural dependencies, (ii) Probabilistic Modeling to relax the discrete constraints, and (iii) Stochastic Neighborhood Perturbation (SNP) to foster solution diversity and ensure effective exploration. This integrated design allows the model to effectively navigate the solution space and converge to high-quality solutions.

GNN-based Parameterization. We employ a Graph Neural Network (GNN), denoted as f_θ , to generate the probability vector $\hat{x}$. The ILP is first represented as a bipartite graph $\mathcal{G}$, and the GNN processes this structure to output $\hat{x} = f_\theta(\mathcal{G})$. The GNN is utilized to capture the complex relational structure of ILP constraints, as detailed in Appendix B.2.

Probabilistic Modeling. We relax the original discrete ILP into a continuous formulation by introducing a probability vector $\hat{x} \in [0, 1]^n$. This vector parameterizes a multivariate Bernoulli distribution, $p(\cdot | \hat{x})$, which transforms the optimization task into a search for parameters that yield feasible solutions with low objective costs.

Stochastic Neighborhood Perturbation (SNP). To increase the diversity of sampled solutions, we introduce the Stochastic Neighborhood Perturbation (SNP) operator. During training, we first sample a solution x from the GNN’s predicted distribution and then apply the SNP operator to obtain a perturbed sample x' . The perturbation magnitude is controlled by an annealed parameter ρ_t , balancing broad exploration early in training with fine-tuning later. The GNN parameters θ are optimized based on these perturbed samples, as discussed in Section 4.2.

Inference. Once trained, the GNN can swiftly produce high-quality solutions for unseen ILP instances by sampling from its learned

distribution $p(\cdot | f_\theta(\mathcal{G}_i))$. Furthermore, these solutions can be leveraged to enhance traditional solvers. We employ a local branching technique to warm-start a conventional solver, with details provided in Appendix B.3. This two-pronged inference approach allows PUMA to function both as a fast heuristic and as a powerful accelerator for exact methods.

4.2 Stochastic Neighborhood Perturbation-Based Probabilistic Modeling

To tackle the combinatorial nature of ILP, as defined in Problem 1, a common strategy is to relax the binary constraints. Inspired by Geng et al. [11], we model each variable x_i with a Bernoulli distribution parameterized by $\hat{x} \in [0, 1]^n$. To keep the objective scalar-valued, we introduce the aggregate violation function

$$g(x) \triangleq \sum_{j=1}^m \max(A_j x - b_j, 0), \quad (3)$$

which measures the total positive constraint violation of x . The ILP is then transformed into an unsupervised learning problem of finding $\hat{x}$ that minimizes the expected objective value plus a penalty on expected violation:

$$\min_{\hat{x} \in [0, 1]^n} c^\top \hat{x} + \mu \mathbb{E}_{x \sim p(\cdot | \hat{x})} [g(x)], \quad (4)$$

where $p(\cdot | \hat{x})$ is the joint Bernoulli distribution and $\mu > 0$ is a penalty coefficient.

To theoretically justify this formulation, we introduce an intermediate probabilistic reformulation that enforces the constraints as a single scalar expectation:

$$\min_{\hat{x} \in [0, 1]^n} c^\top \hat{x} \quad \text{s.t.} \quad \mathbb{E}_{x \sim p(\cdot | \hat{x})} [g(x)] = 0. \quad (5)$$

This formulation bridges the discrete ILP and our unconstrained learning objective. We now establish that optimizing the penalized objective in Problem 4 allows for the exact constructive recovery of an optimal solution to the original ILP.

LEMMA 1 (CONSTRUCTIVE RECOVERY). *If the original ILP (Problem 1) is feasible, then there exists a finite threshold $\mu^* > 0$ such that for any penalty parameter $\mu > \mu^*$, an exact optimal binary solution to Problem 1 can be directly constructed from any optimal solution to Problem 4 without any additional search. This is established by the following two properties:*

- (i) Let $I_C = \{i \in [n] : c_i \neq 0\}$. Suppose $\hat{x}^* \in [0, 1]^n$ is an optimal solution to Problem 5. Then $\hat{x}_i^* \in \{0, 1\}$ for all $i \in I_C$, and the strict feasibility of $\hat{x}^*$ guarantees that every binary vector $x^* \in \text{supp } p(\cdot | \hat{x}^*)$ is feasible for Problem 1 with $c^\top x^* = c^\top \hat{x}^*$. Letting $I_{\hat{x}^*} = \{i \in [n] : \hat{x}_i^* \in \{0, 1\}\}$, any $x^* \in \{0, 1\}^n$ satisfying $x_i^* = \hat{x}_i^*$ for all $i \in I_{\hat{x}^*}$ is therefore an optimal solution to Problem 1.
- (ii) There exists a finite threshold $\mu^* > 0$ such that for any $\mu > \mu^*$, any optimal solution to Problem 4 is also an optimal solution to Problem 5.

While theoretically sound, this probabilistic relaxation can suffer from a rapid loss of solution diversity. As the predicted probabilities $\hat{x}$ prematurely converge to 0 or 1, the diversity of sampled solutions diminishes, hindering exploration and eventually entrapping the model in suboptimal local minima.

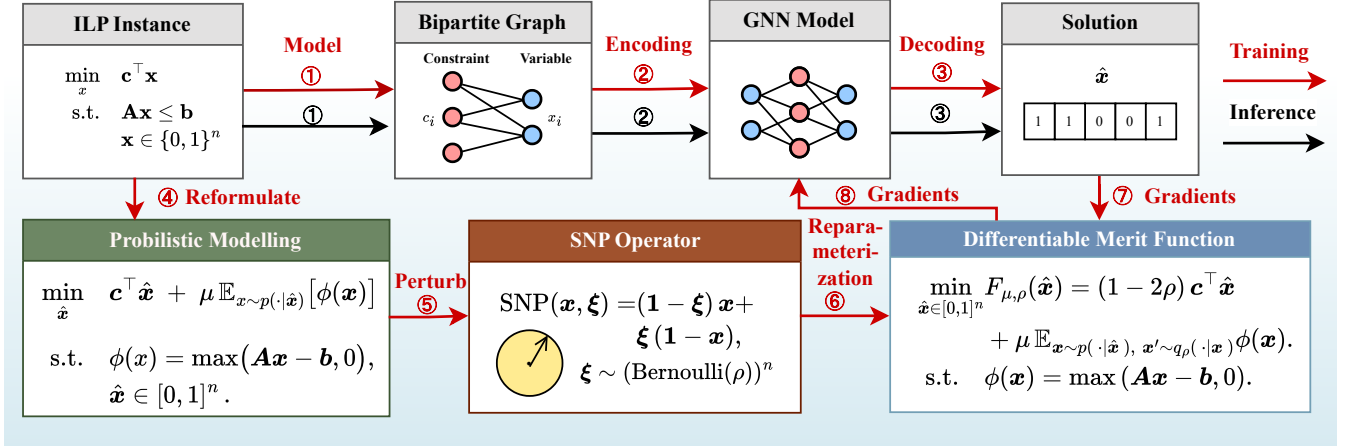


Figure 2: An overview of the PUMA framework. The training pipeline (red arrows) learns a unsupervised learning scheme by integrating a GNN-based parameterization, a probabilistic relaxation of the ILP, and a stochastic neighborhood perturbation (SNP) operator to ensure robust exploration. The inference pipeline (black arrows) then uses the trained GNN to rapidly generate high-quality solutions for new ILP instances.

To address this, we introduce the stochastic neighborhood perturbation (SNP) operator. Instead of sampling directly from $p(\cdot | \hat{x})$, we first draw a sample x and then perturb it to a nearby solution x' by flipping a controlled number of its bits. We define the SNP operator as:

$$\text{SNP}(x, \xi) = (1 - \xi)x + \xi(1 - x), \quad (6)$$

where $\xi \in \{0, 1\}^n$ is a random mask drawn from a Bernoulli distribution, $\xi \sim (\text{Bernoulli}(\rho))^n$. The parameter ρ , termed the perturbation magnitude, is the probability of flipping each variable. This results in an expected Hamming distance of $\mathbb{E}[\text{Hamming}(\text{SNP}(x, \xi), x)] = n\rho$.

By incorporating this operator, we define a new perturbed distribution $q_\rho(x' | x)$ and modify the learning objective to account for the perturbation.

THEOREM 2 (SURROGATE MERIT FUNCTION EQUIVALENCE). *The optimization problem defined by the perturbed objective is equivalent to minimizing the surrogate merit function $F_{\mu, \rho}(\hat{x})$,*

$$\min_{\hat{x} \in [0, 1]^n} \left\{ F_{\mu, \rho}(\hat{x}) = (1 - 2\rho)c^\top \hat{x} + \mu \mathbb{E}_{x \sim p(\cdot | \hat{x}), x' \sim q_\rho(\cdot | x)} [g(x')] \right\}, \quad (7)$$

where $g(\cdot)$ is the aggregate violation defined in Equation 3.

By leveraging standard reparameterization techniques (e.g., Gumbel-Softmax), we can optimize this objective with respect to $\hat{x}$ in an end-to-end manner.

To facilitate early-stage exploration and late-stage refinement, we dynamically control the perturbation magnitude ρ using an exponentially decaying schedule. We start with an initial magnitude ρ_0 and decay it over training iterations t with a factor $\alpha \in (0, 1)$:

$$\rho_t = \rho_0 \alpha^t \quad (8)$$

This allows for broad exploration in the early stages and fine-tuning towards the end, ensuring the model converges to a high-quality solution.

For implementation, we train the GNN parameters θ by minimizing the surrogate merit function from Equation 7. For a batch of ILP instances $\mathcal{D} = \{\mathcal{G}_1, \dots, \mathcal{G}_M\}$, the loss is estimated via a nested Monte Carlo scheme. For each instance $\mathcal{G}_i$ at training step t :

- (i) We sample K base solutions $\{x_i^{(k)}\}_{k=1}^K$ from the base distribution $p(\cdot | f_\theta(\mathcal{G}_i))$ using the Gumbel-Softmax trick.
- (ii) For each base solution $x_i^{(k)}$, we sample M perturbed solutions $\{x_i'^{(k, m)}\}_{m=1}^M$ from the perturbed distribution $q_{\rho_t}(\cdot | x_i^{(k)})$, where ρ_t is the annealed perturbation magnitude.

The training loss for the batch is the average of the estimated merit function, where the expectation is approximated over all $K \times M$ perturbed samples:

$$\mathcal{L}(\theta; \mathcal{D}) = \frac{1}{|\mathcal{D}|} \sum_{i=1}^{|\mathcal{D}|} \left[(1 - 2\rho_t) c_i^\top f_\theta(\mathcal{G}_i) + \frac{\mu}{KM} \sum_{k=1}^K \sum_{m=1}^M g_i(x_i'^{(k, m)}) \right]. \quad (9)$$

4.3 Theoretical Analysis

To provide a deeper understanding of our proposed method, we analyze the SNP operator from a theoretical standpoint. We demonstrate that our surrogate objective function (Equation 7) is equivalent to optimizing the original probabilistic objective (Equation 4) augmented with a dynamic, exploration-guiding regularization term. This regularization term, which we denote $\mathcal{R}_\rho(\hat{x})$, functionally regularizes the training process by smoothing the loss landscape and penalizing “brittle” solutions that are sensitive to small

perturbations. As we will show, this term encourages robust exploration and vanishes as training progresses, ensuring convergence to the intended objective.

Recall the scalar violation function $g(\mathbf{x}) = \sum_{j=1}^m \max(A_j \mathbf{x} - b_j, 0)$ from Equation 3, and let $p(\cdot | \hat{\mathbf{x}})$ be the base Bernoulli distribution parameterized by $\hat{\mathbf{x}}$. Our analysis is structured around the following results.

THEOREM 3 (DECOMPOSITION OF THE SURROGATE OBJECTIVE). *The SNP-based surrogate merit function $F_{\mu, \rho}(\hat{\mathbf{x}})$ from Equation 7 can be expressed as a sum of the scaled original probabilistic objective and a dynamic regularization term $\mathcal{R}_\rho(\hat{\mathbf{x}})$:*

$$F_{\mu, \rho}(\hat{\mathbf{x}}) = (1 - 2\rho) \mathbf{c}^\top \hat{\mathbf{x}} + \mu \mathbb{E}_{\mathbf{x} \sim p(\cdot | \hat{\mathbf{x}})} [g(\mathbf{x})] + \mu \mathcal{R}_\rho(\hat{\mathbf{x}}), \quad (10)$$

where the regularization term $\mathcal{R}_\rho(\hat{\mathbf{x}})$, which represents the expected change in constraint violation induced by the SNP operator, is defined as:

$$\mathcal{R}_\rho(\hat{\mathbf{x}}) := \mathbb{E}_{\mathbf{x} \sim p(\cdot | \hat{\mathbf{x}})} \left[\mathbb{E}_{\mathbf{x}' \sim q_\rho(\cdot | \mathbf{x})} [g(\mathbf{x}')] - g(\mathbf{x}) \right]. \quad (11)$$

This term effectively measures the robustness of solutions sampled from $p(\cdot | \hat{\mathbf{x}})$, penalizing those whose neighbors are likely to have high constraint violations.

This theorem formally establishes the structure of our surrogate objective. The next result shows that the influence of the exploration-guiding term is temporary and controlled by our annealing schedule.

THEOREM 4 (ASYMPTOTIC CONSISTENCY). *As the perturbation magnitude ρ approaches zero, the regularization term vanishes, and the SNP-based objective $F_{\mu, \rho}(\hat{\mathbf{x}})$ converges to the original probabilistic objective from Problem 4. Specifically:*

$$\lim_{\rho \rightarrow 0} \mathcal{R}_\rho(\hat{\mathbf{x}}) = 0, \quad (12)$$

and consequently,

$$\lim_{\rho \rightarrow 0} F_{\mu, \rho}(\hat{\mathbf{x}}) = \mathbf{c}^\top \hat{\mathbf{x}} + \mu \mathbb{E}_{\mathbf{x} \sim p(\cdot | \hat{\mathbf{x}})} [g(\mathbf{x})]. \quad (13)$$

The next theorem quantifies the rate at which $\mathcal{R}_\rho$ vanishes.

While Theorem 4 guarantees eventual convergence to the correct objective, its practical utility hinges on the rate of this convergence. The following theorem quantifies this rate, showing that the regularization term's influence diminishes gracefully and linearly with ρ .

THEOREM 5 (LINEAR BOUND ON THE REGULARIZATION TERM). *For the ILP formulations in our work, the violation function $g(\mathbf{x})$ is bounded because the decision variables are binary. A concrete upper bound is obtained by summing the worst-case positive violation across all m constraints: $M = \sum_{j=1}^m \max(0, \sum_{i: A_{ji} > 0} A_{ji} - b_j)$. Given this, the magnitude of the regularization term $\mathcal{R}_\rho(\hat{\mathbf{x}})$ is linearly bounded by the perturbation magnitude ρ . Specifically, the regularization term vanishes at a rate of $\mathcal{O}(\rho)$:*

$$|\mathcal{R}_\rho(\hat{\mathbf{x}})| \leq M n \rho. \quad (14)$$

In summary, our theoretical analysis confirms that the SNP operator induces a well-behaved landscape-smoothing effect, captured by the regularization term $\mathcal{R}_\rho(\hat{\mathbf{x}})$. This term guides the model away from sharp local minima towards broader, more robust regions of the solution space. The effect is strongest in the early stages

of training to promote exploration and systematically diminishes according to the annealing schedule, guiding the model to optimize the intended ILP objective by the end of training.

5 Experiments

This section presents a comprehensive empirical evaluation of PUMA. We begin by detailing the experimental setup in Section 5.1. Subsequently, we assess PUMA's performance in terms of solution quality (Section 5.2), solver acceleration (Section 5.3), generalization to larger problems (Section 5.4), optimization behaviors (Section 5.5). Finally, we conduct sensitivity analysis on a key component of our framework (Section 5.6). Further experimental results and implementation details are provided in Appendices D and B, respectively.

5.1 Experimental Setup

Benchmark. We evaluate our approach on three widely-used ILP benchmarks: Combinatorial Auction (CA), Maximum Independent Set (IS), and Set Covering (SC). For SC, we adopt the dataset from Geng et al. [11], which contains instances with 1,000 columns and 2,000 rows. For CA, we generate instances with 500 items and 2,500 bids. For IS, we construct graphs with 2,000 nodes and an affinity of 5. All other generation parameters follow the settings of Gasse et al. [9]. Consistent with previous work [11, 16, 27], we partition each dataset into 240 training, 60 validation, and 100 test instances. These benchmarks were chosen for their prevalence in the literature [9, 16, 45] and the significant challenges they pose to state-of-the-art solvers. Detailed dataset descriptions are in Appendix C.

Baseline. We benchmark PUMA against several representative baselines to highlight its advantages: (i) traditional ILP solvers, namely SCIP and Gurobi; (ii) the Predict-and-Search (PS) framework [16], a supervised learning approach; (iii) the DiffILO framework [11], an unsupervised learning approach. We do not include Neural Diving (ND) [9] as a baseline because its prediction component is identical to that of PS, and it was shown to have weaker performance than PS by Han et al. [16]. Further baseline comparisons are available in Section D.6.

Metric. We evaluate performance using three primary metrics: the final objective value, the absolute primal gap, and the relative primal gap. Our analysis relies on a Best-Known Solution (BKS) with objective value $f(x^*)$ [2]. We adopt a *uniform* BKS protocol: for every benchmark instance, the BKS is defined as the best objective value found by *any* evaluated method (including all baselines and PUMA variants). Concretely, we first run Gurobi with a 3,600-second reference budget to establish a strong baseline, then evaluate all methods under a strictly shorter 1,000-second time limit, and finally take the BKS as the best objective across the entire pool. Following prior work [11, 16, 27], on CA and SC the 3,600-second Gurobi reference is not surpassed and therefore serves as the BKS; on IS, PUMA + Gurobi finds superior solutions within 1,000 seconds, which then become the BKS. This single rule is applied identically across tasks, ensuring that all reported primal gaps are computed against a common, uniformly defined reference.

Given a solution $\mathbf{x}_t$ with objective value $f(\mathbf{x}_t)$ at time t , the absolute primal gap is $\gamma_{\text{abs}}(t) = |f(\mathbf{x}_t) - f(\mathbf{x}^*)|$. We also report

the relative primal gap (γ_{rel}), which normalizes this gap by the magnitude of the objective values.

Configuration. Model training was conducted on an NVIDIA GeForce RTX 4070 GPU with 12 GB of memory. All evaluations were run on a workstation equipped with a 12th Gen Intel Core i5-12400 CPU and 64 GB of RAM, using Gurobi 11.0.1 and SCIP 5.4.0. During testing, the solver time limit was set to 1,000 seconds, consistent with Han et al. [16]; the 3,600-second Gurobi run is used *only* to establish the BKS reference and never as a competing method under the evaluation budget. Additional implementation details are provided in Appendix B.

Table 1: Average objective values and absolute primal gaps (Gap) for solutions generated directly by each method on 100 test instances, without solver enhancement. “Gurobi_{init}” represents Gurobi’s built-in heuristic and “DiffILO-L2” is a variant of DiffILO with L2 regularization.

Method	CA (BKS: 37011)		IS (BKS: 838)		SC (BKS: 86)	
	Obj $\uparrow$	Gap $\downarrow$	Obj $\uparrow$	Gap $\downarrow$	Obj $\downarrow$	Gap $\downarrow$
Gurobi _{init}	26576	10436	493	345	2404	2318
DiffILO	29523	7489	803	34	123	36
DiffILO-L2	29818	7194	786	52	124	37
PUMA	33690	3322	822	15	120	33
Improve	55.14%		55.64%		8.44%	

5.2 Performance on Solution Prediction

We first evaluate the quality of the solutions generated by PUMA without any subsequent solver refinement. We compare PUMA with Gurobi’s built-in heuristic, the state-of-the-art unsupervised method DiffILO, and a variant, DiffILO-L2, which incorporates L2 regularization to prevent overfitting. Further studies in Appendix D.1 confirm PUMA’s superior performance.

Table 1 presents the average objective values and absolute primal gaps (Gap) across 100 test instances. The results demonstrate that PUMA produces the highest-quality solutions across all benchmarks. Compared to the next-best unsupervised method, DiffILO, PUMA reduces the absolute primal gap by a remarkable 55.14% on CA, 55.64% on IS, and 8.44% on SC. This highlights the superiority of our proposed perturbation strategy in guiding the model toward more promising regions of the solution space.

5.3 Performance on Improving Traditional Solvers

A key application of learning-based methods is to accelerate traditional solvers by providing a high-quality initial solution. To evaluate PUMA’s ability to accelerate traditional solvers, we use the solution generated by PUMA to guide traditional solvers and measure performance over a 1,000-second time limit.

Figure 3 shows the average primal gap over time when using Gurobi. On the CA benchmark, PUMA +Gurobi consistently outperforms all baselines. While not leading from the outset on the IS and

SC tasks, PUMA steadily closes the primal gap, ultimately surpassing all baselines by a significant margin.

Table 2 reports the final objective values and absolute primal gaps at the 1,000-second mark. PUMA achieves the best results across all benchmarks, even finding zero-gap solutions for IS and SC. Notably, on the IS benchmark, PUMA +Gurobi finds a solution matching the BKS, a result that standalone Gurobi fails to achieve even with 3,600 seconds. These results not only demonstrate the superiority of PUMA but also underscore the effectiveness of unsupervised learning approaches, which are competitive with the supervised method. Additional evaluations, including results for the SCIP solver and the primal integral analysis, are provided in Appendix D.3.

5.4 Generalization Analysis

To evaluate the scalability and generalization capabilities of PUMA, we conduct experiments on two distinct types of datasets: larger-scale synthetic instances and challenging real-world problems from MIPLIB 2017. Results are presented in Figure 4. Further statistical results are provided in Appendices D.5 and D.4.

Scalability. First, we test the model’s performance on larger versions of the previous benchmarks (CA, IS and SC). We perform evaluations on these instances using the models trained on the original smaller datasets, without any retraining. Detailed parameters and problem sizes for these large-scale instances are provided in Appendix C.2.

Real-world Generalization on MIPLIB. To further demonstrate the effectiveness of PUMA on realistic datasets, we conduct additional experiments on the MIPLIB 2017 benchmark [12]. Since learning directly on the full MIPLIB is challenging due to its diversity, we construct a CVS subset following previous work [42], which consists of capacitated vertex separator (CVS) problem instances.

5.5 Interpretability Analysis

To empirically interpret the learning dynamics and the efficacy of the SNP mechanism, we analyze a toy ILP instance:

$$\min_{x_1, x_2 \in \{0,1\}} \{-11x_1 - 10x_2 \mid x_1 + x_2 \leq 1\}. \quad (15)$$

This instance presents a non-trivial challenge due to the proximity of the global optimum (1, 0) and the sub-optimal local minimum (0, 1). As visualized in Figure 5a, the discrete constraint creates a rugged landscape where the sub-optimal basin of attraction can easily entrap the optimization process. We address this by transforming the problem into a continuous form with penalty terms:

$$F_\mu(\theta) = \mathbb{E}_{x \sim p_\theta} [-11x_1 - 10x_2 + \mu \cdot \max(x_1 + x_2 - 1, 0)], \quad (16)$$

where μ is the penalty coefficient. As shown in Figure 5b, the baseline method, which relies solely on optimizing this surrogate objective, rapidly converges to the sub-optimal solution (0, 1) and becomes trapped, failing to explore the landscape effectively.

In contrast, PUMA incorporates the SNP mechanism. We quantify the optimization behavior using *hit probability*, defined as the probability of sampling the global optimal solution within the current batch. The dynamics are illustrated in Figure 5d. Initially, the hit probability for both methods decreases, indicating that the optimization trajectories are being pulled towards the local basin of

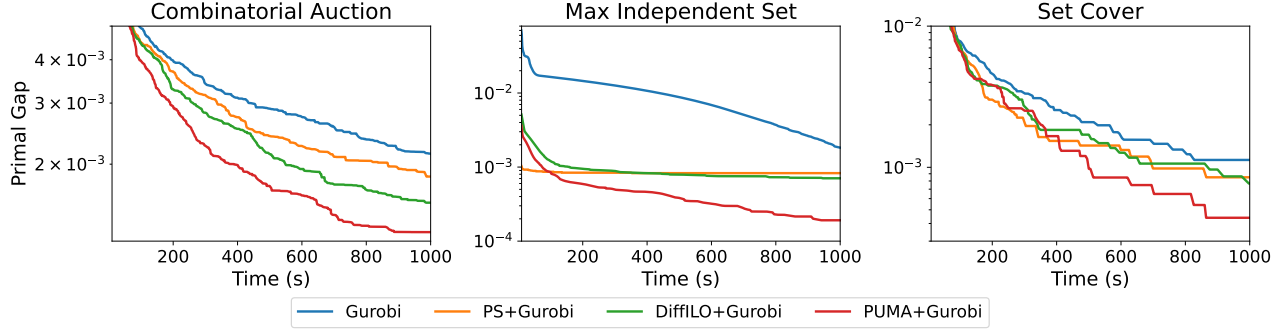


Figure 3: Average relative primal gap over time of different approaches across 100 test instances. A lower curve indicates better performance. The PUMA +Gurobi approach consistently achieves a lower final primal gap, demonstrating its ability to find higher-quality solutions within the time limit.

Table 2: Average objective and absolute primal gap (Gap_{abs}) after 1,000 seconds on 100 test instances. Best results are in bold. The “Improvement” row highlights the percentage reduction in Gap_{abs} by PUMA +Gurobi compared to the standalone Gurobi solver.

Method	CA (BKS: 37011.96)		IS (BKS: 837.67)		SC (BKS: 86.47)	
	Obj $\uparrow$	Gap $_{\text{abs}}\downarrow$	Obj $\uparrow$	Gap $_{\text{abs}}\downarrow$	Obj $\downarrow$	Gap $_{\text{abs}}\downarrow$
Gurobi	36968.19	43.77	836.30	1.37	86.53	0.06
PS+Gurobi	36979.43	32.53	837.14	0.53	86.51	0.04
DiffiLO+Gurobi	36990.19	21.77	837.24	0.43	86.50	0.03
PUMA +Gurobi	37000.38	11.58	837.67	0.00	86.47	0.00
Improvement		46.8%		100.00%		100.00%

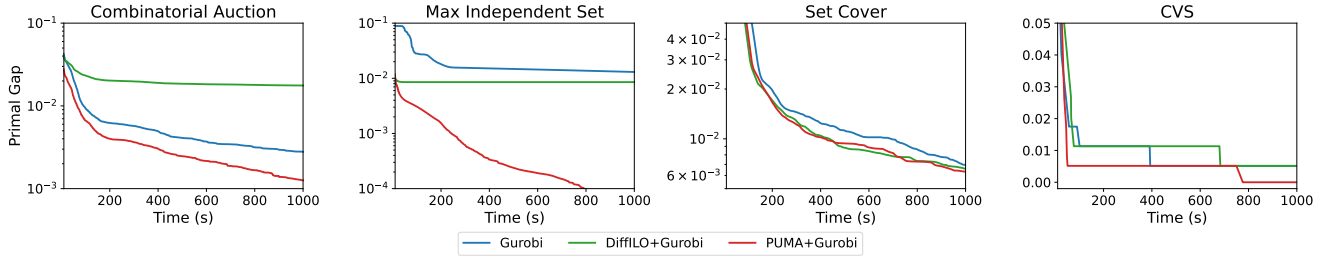


Figure 4: Generalization performance analysis on both synthetic and real-world datasets. The curves illustrate the average relative primal gap over time for: (1) larger-scale instances (CA, IS, SC), and (2) the CVS subset from MIPLIB 2017, demonstrating effectiveness on realistic problems.

attraction. However, unlike the baseline which continues to degrade or stagnate, PUMA leverages SNP to introduce necessary exploratory variance. This allows PUMA to sample solutions containing informative directions pointing towards the global optimum. Consequently, instead of remaining trapped, the hit probability for PUMA reverses its downward trend and increases significantly, ultimately allowing the model to escape the local minimum and converge to the global optimum (1, 0). To further quantify this robustness, we conducted 50 independent runs, in which PUMA successfully converged to the global optimum in every single trial.

5.6 Sensitivity Analysis

We conduct sensitivity analysis to evaluate the robustness of PUMA’s perturbation mechanism. Specifically, we analyze the impact of two critical hyperparameters: the initial perturbation magnitude ρ_0 and the decay schedule of the perturbation magnitude. Figure 6 visualizes the training dynamics under different configurations. Further ablation studies are presented in Section D.7.

Sensitivity to Initial Perturbation Magnitudes. We first assess the model’s sensitivity to the initial perturbation magnitudes ρ_0 . As

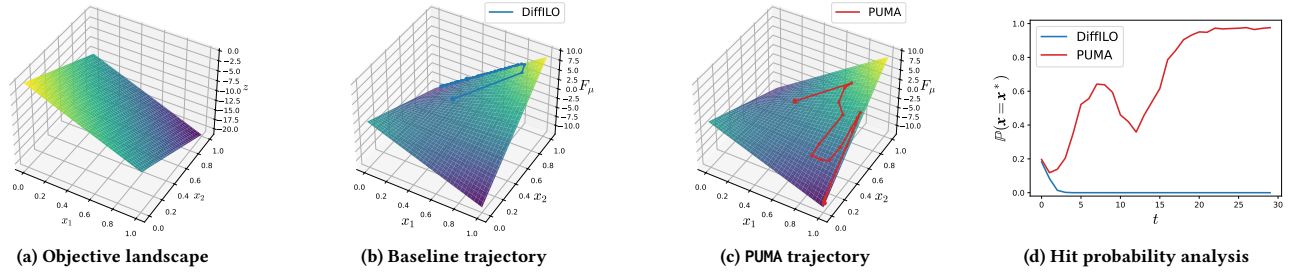


Figure 5: Visualization of optimization dynamics on the toy ILP instance. (a) The landscape of the unconstrained linear objective function. (b) The baseline method (DiffILO), relying solely on a surrogate objective, becomes trapped in the basin of attraction of the sub-optimal solution $(0, 1)$. (c) In contrast, PUMA leverages stochastic perturbation to explore the landscape, successfully converging to the global optimum $(1, 0)$. (d) The hit probability (probability of sampling the global optimum) over learning steps, demonstrating that PUMA effectively recovers from initial degradation and stably converges to the optimal solution.

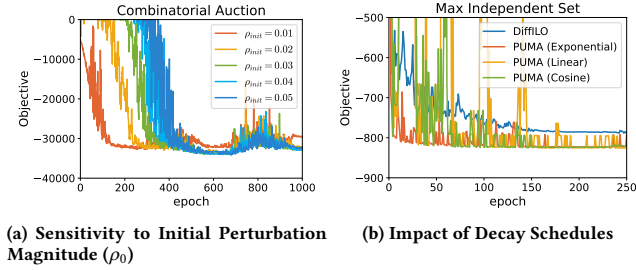


Figure 6: Ablation studies on the SNP mechanism. (a) Validation objective on the CA benchmark with varying initial perturbation magnitudes ρ_0 . (b) Convergence comparison of different decay schedules on the IS benchmark.

shown in Figure 6a, PUMA exhibits strong robustness across a reasonable range of ρ_0 . While extremely small values provide limited exploratory guidance and excessively large values may hinder convergence, the model maintains high performance for ρ_0 values between 0.02 and 0.05. This suggests that PUMA does not require hyper-specific tuning of this parameter to achieve superior results.

Impact of Decay Schedules. We further investigate the influence of the decay schedule for the perturbation magnitude ρ (i.e., the annealing temperature). We compare linear, cosine, and exponential decay schedules on the IS benchmark. Figure 6b indicates that while all schedules allow PUMA to surpass the DiffILO baseline, the exponential schedule yields the superior convergence profile. Unlike the linear and cosine schedules, the exponential schedule ensures a rapid yet smooth descent. This balance effectively facilitates broad exploration in the early stages and precise refine in later stages, avoiding the instability observed in other approaches.

6 Conclusion

In this paper, we introduce PUMA, a novel unsupervised learning method designed to address the critical challenge of entrapment in

local minima in unsupervised approaches to solving ILP. At the core of PUMA is a stochastic neighborhood perturbation (SNP) operator. Guided by an exponentially decaying schedule, this operator transitions the learning dynamics from comprehensive exploration of the solution space to fine-grained refinement, effectively preventing entrapment in suboptimal local minima. Extensive experiments on challenging benchmarks demonstrate that PUMA not only yields higher-quality solutions to accelerate solvers like Gurobi and SCIP, but also robustly generalizes to large-scale and realistic problems. Future work includes developing adaptive perturbation schedules and extending the methodology to other problem classes, such as mixed-integer linear programs.

Acknowledgments

This study was supported by the National Natural Science Foundation of China (NSFC, 72571284, 72421002, 72595861), Hunan Provincial Fund for Distinguished Young Scholars (2025JJ20073), Science and Technology Innovation Program of Hunan Province (2023RC3009), Innovation Research Foundation of National University of Defense Technology (JS24-05), and the Major Science and Technology Projects in Changsha, China (kq2301008).

References

- [1] Maria-Florina Balcan, Travis Dick, Tuomas Sandholm, and Ellen Vitercik. 2018. Learning to branch. In *International conference on machine learning*. PMLR, 344–353.
- [2] Timo Berthold. 2006. *Primal heuristics for mixed integer programs*. Ph.D. Dissertation. Zuse Institute Berlin (ZIB).
- [3] Fanchen Bu, Hyeonsoo Jo, Soo Yong Lee, Sungsoo Ahn, and Kijung Shin. 2024. Tackling prevalent conditions in unsupervised combinatorial optimization: Cardinality, minimum, covering, and more. *arXiv preprint arXiv:2405.08424* (2024).
- [4] Junyang Cai, Taoan Huang, and Bistra Dilkina. 2024. Learning Backdoors for Mixed Integer Linear Programs with Contrastive Learning. In *ECAI 2024*. IOS Press, 2418–2425.
- [5] Joost T. de Kruijff, Cor A. J. Hurkens, and Ton G. de Kok. 2018. Integer programming models for mid-term production planning for high-tech low-volume supply chains. *European Journal of Operational Research* 269, 3 (2018), 984–997.
- [6] Brian T. Denton, John Forrest, and R. John Milne. 2006. IBM solves a mixed-integer program to optimize its semiconductor supply chain. *Interfaces* 36, 5 (2006), 386–399.
- [7] Jian-Ya Ding, Chao Zhang, Lei Shen, Shengyin Li, Bing Wang, Yinghui Xu, and Le Song. 2020. Accelerating primal solution findings for mixed integer programs

- based on solution prediction. In *Proceedings of the aaai conference on artificial intelligence*, Vol. 34. 1452–1459.
- [8] Matteo Fischetti and Andrea Lodi. 2003. Local branching. *Mathematical programming* 98, 1 (2003), 23–47.
 - [9] Maxime Gasse, Didier Chételat, Nicola Ferroni, Laurent Charlin, and Andrea Lodi. 2019. Exact combinatorial optimization with graph convolutional neural networks. *Advances in neural information processing systems* 32 (2019).
 - [10] Yonatan Geifman and Ran El-Yaniv. 2019. Selectivenet: A deep neural network with an integrated reject option. In *International conference on machine learning*. PMLR, 2151–2159.
 - [11] Zijie Geng, Jie Wang, Xijun Li, Fangzhou Zhu, Jianye Hao, Bin Li, and Feng Wu. 2025. Differentiable integer linear programming. In *The Thirteenth International Conference on Learning Representations*.
 - [12] Ambros Gleixner, Gregor Hendel, Gerald Gamrath, Tobias Achterberg, Michael Bastubbe, Timo Berthold, Philipp Christophel, Kati Jarck, Thorsten Koch, Jeff Linderoth, et al. 2021. MIPLIB 2017: data-driven compilation of the 6th mixed-integer programming library. *Mathematical Programming Computation* 13, 3 (2021), 443–490.
 - [13] Ralph Edward Gomory. 1960. An algorithm for the mixed integer problem. *Report No. P-1885, The Rand Corporation, Santa Monica, CA.* (1960).
 - [14] Prateek Gupta, Maxime Gasse, Elias Khalil, Pawan Mudigonda, Andrea Lodi, and Yoshua Bengio. 2020. Hybrid models for learning to branch. *Advances in neural information processing systems* 33 (2020), 18087–18097.
 - [15] Nicholas G. Hall and Zhixin Liu. 2010. Capacity allocation and scheduling in supply chains. *Operations Research* 58, 6 (2010), 1711–1725.
 - [16] Qingyu Han, Linxin Yang, Qian Chen, Xiang Zhou, Dong Zhang, Akang Wang, Ruoyu Sun, and Xiaodong Luo. 2022. A gnn-guided predict-and-search framework for mixed-integer linear programming. *arXiv preprint arXiv:2302.05636* (2022).
 - [17] He He, Hal Daume III, and Jason M Eisner. 2014. Learning to search in branch and bound algorithms. *Advances in neural information processing systems* 27 (2014).
 - [18] Taoan Huang, Aaron M Ferber, Yuandong Tian, Bistra Dilkina, and Benoit Steiner. 2023. Searching large neighborhoods for integer linear programs with contrastive learning. In *International conference on machine learning*. PMLR, 13869–13890.
 - [19] Taoan Huang, Aaron M Ferber, Arman Zharmagambetov, Yuandong Tian, and Bistra Dilkina. 2024. Contrastive predict-and-search for mixed integer linear programs. In *Forty-first International Conference on Machine Learning*.
 - [20] Zeren Huang, Kerong Wang, Furui Liu, Hui-Ling Zhen, Weinan Zhang, Mingxuan Yuan, Jianye Hao, Yong Yu, and Jun Wang. 2022. Learning to select cuts for efficient mixed-integer programming. *Pattern Recognition* 123 (2022), 108353.
 - [21] Nikolaos Karalias and Andreas Loukas. 2020. Erdos goes neural: an unsupervised learning framework for combinatorial optimization on graphs. *Advances in Neural Information Processing Systems* 33 (2020), 6659–6672.
 - [22] Elias Khalil, Pierre Le Bodic, Le Song, George Nemhauser, and Bistra Dilkina. 2016. Learning to branch in mixed integer programming. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 30.
 - [23] Nicklas Klein, Mario Gnägi, and Norbert Trautmann. 2024. Mixed-integer linear programming for project scheduling under various resource constraints. *European Journal of Operational Research* 319, 1 (2024), 79–88.
 - [24] Yufei Kuang, Jie Wang, Haoyang Liu, Fangzhou Zhu, Xijun Li, Jia Zeng, Jianye Hao, Bin Li, and Feng Wu. 2024. Rethinking branching on exact combinatorial optimization solver: The first deep symbolic discovery framework. In *The Twelfth International Conference on Learning Representations*.
 - [25] AH Land and AG Doig. 1960. An Automatic Method of Solving Discrete Programming Problems. *Econometrica* 28, 3 (1960), 497–520.
 - [26] Haotian Ling, Zhihai Wang, and Jie Wang. 2024. Learning to stop cut generation for efficient mixed-integer linear programming. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 20759–20767.
 - [27] Haoyang Liu, Jie Wang, Zijie Geng, Xijun Li, Yuxuan Zong, Fangzhou Zhu, Jianye Hao, and Feng Wu. 2025. Apollo-MILP: An alternating prediction-correction neural solving framework for mixed-integer linear programming. *arXiv preprint arXiv:2503.01129* (2025).
 - [28] Vinod Nair, Sergey Bartunov, Felix Gimeno, Ingrid Von Glehn, Pawel Lichocki, Ivan Lobov, Brendan O’Donoghue, Nicolas Sonnerat, Christian Tjandraatmadja, Pengming Wang, et al. 2020. Solving mixed integer programs using neural networks. *arXiv preprint arXiv:2012.13349* (2020).
 - [29] Max B Paulus, Giulia Zarpellon, Andreas Krause, Laurent Charlin, and Chris Maddison. 2022. Learning to cut by looking ahead: Cutting plane selection via imitation learning. In *International conference on machine learning*. PMLR, 17584–17600.
 - [30] Krzysztof Postek and Dick den Hertog. 2016. Multistage adjustable robust mixed-integer optimization via iterative splitting of the uncertainty set. *INFORMS Journal on Computing* 28, 3 (2016), 553–574.
 - [31] Tianle Pu, Chao Chen, Li Zeng, Shixuan Liu, Rui Sun, and Changjun Fan. 2024. Solving combinatorial optimization problem over graph through qubo transformation and deep reinforcement learning. In *2024 IEEE International Conference on Data Mining (ICDM)*. IEEE, 390–399.
 - [32] Tianle Pu, Changjun Fan, Mutian Shen, Yizhou Lu, Li Zeng, Zohar Nussinov, Chao Chen, and Zhong Liu. 2024. Exploratory Combinatorial Optimization Problem Solving via Gauge Transformation. In *2024 IEEE International Conference on Data Mining (ICDM)*. IEEE, 821–826.
 - [33] Tianle Pu, Zijie Geng, Haoyang Liu, Shixuan Liu, Jie Wang, Li Zeng, Chao Chen, and Changjun Fan. 2026. RoME: Domain-robust mixture-of-experts for MILP solution prediction across domains. *Advances in neural information processing systems* 38 (2026), 33633–33661.
 - [34] Tianle Pu, Jianing Li, Yingying Gao, Shixuan Liu, Zijie Geng, Haoyang Liu, Chao Chen, and Changjun Fan. 2026. Coco-milp: Inter-variable contrastive and intra-constraint competitive milp solution prediction. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 40. 24882–24890.
 - [35] Pol Puigdemont, Stratis Skoulakis, Grigorios Chrysos, and Volkan Cevher. 2024. Learning to remove cuts in integer linear programming. *arXiv preprint arXiv:2406.18781* (2024).
 - [36] Ruizhong Qiu, Zhiqing Sun, and Yiming Yang. 2022. Dimes: A differentiable meta solver for combinatorial optimization problems. *Advances in Neural Information Processing Systems* 35 (2022), 25531–25546.
 - [37] Martin JA Schuetz, J Kyle Brubaker, and Helmut G Katzgraber. 2022. Combinatorial optimization with physics-inspired graph neural networks. *Nature Machine Intelligence* 4, 4 (2022), 367–377.
 - [38] Paula Metzker Soares, Simon Thevenin, Yossiri Adulyasak, and Alexandre Dolgui. 2024. Adaptive robust optimization for lot-sizing under yield uncertainty. *European Journal of Operational Research* 313, 2 (2024), 513–526.
 - [39] Jialin Song, Yisong Yue, Bistra Dilkina, et al. 2020. A general large neighborhood search framework for solving integer linear programs. *Advances in Neural Information Processing Systems* 33 (2020), 20012–20023.
 - [40] Nicolas Sonnerat, Pengming Wang, Ira Ktena, Sergey Bartunov, and Vinod Nair. 2021. Learning a large neighborhood search algorithm for mixed integer programs. *arXiv preprint arXiv:2107.10201* (2021).
 - [41] Yunhao Tang, Shipra Agrawal, and Yuri Faenza. 2020. Reinforcement learning for integer programming: Learning to cut. In *International conference on machine learning*. PMLR, 9367–9376.
 - [42] Haoyu Wang, Jialin Liu, Xiaohan Chen, Xinshang Wang, Pan Li, and Wotao Yin. 2023. Dig-milp: a deep instance generator for mixed-integer linear programming with feasibility guarantee. *arXiv preprint arXiv:2310.13261* (2023).
 - [43] Haoyu Peter Wang, Nan Wu, Hang Yang, Cong Hao, and Pan Li. 2022. Unsupervised learning for combinatorial optimization with principled objective relaxation. *Advances in Neural Information Processing Systems* 35 (2022), 31444–31458.
 - [44] Yaoxin Wu, Wen Song, Zhiguang Cao, Jie Zhang, Abhishek Gupta, and Mingyan Lin. 2022. Graph learning assisted multi-objective integer programming. *Advances in Neural Information Processing Systems* 35 (2022), 17774–17787.
 - [45] Huigen Ye, Hua Xu, and Hongyan Wang. 2024. Light-MILPopt: Solving Large-scale Mixed Integer Linear Programs with Lightweight Optimizer and Small-scale Training Dataset. In *The Twelfth International Conference on Learning Representations*.
 - [46] Huigen Ye, Hua Xu, Hongyan Wang, Chengming Wang, and Yu Jiang. 2023. GNN&GBDT-guided fast optimizing framework for large-scale integer programming. In *International conference on machine learning*. PMLR, 39864–39878.
 - [47] Kaan Yilmaz and Neil Yorke-Smith. 2021. A study of learning search approximation in mixed integer branch and bound: Node selection in scip. *Ai 2*, 2 (2021), 150–178.
 - [48] Taehyun Yoon. 2022. Confidence threshold neural diving. *arXiv preprint arXiv:2202.07506* (2022).
 - [49] Giulia Zarpellon, Jason Jo, Andrea Lodi, and Yoshua Bengio. 2021. Parameterizing branch-and-bound search trees to learn branching policies. In *Proceedings of the aaai conference on artificial intelligence*, Vol. 35. 3931–3939.
 - [50] Hao Zeng, Jiaqi Wang, Avirup Das, Junying He, Kunpeng Han, Haoyuan Hu, and Mingfei Sun. 2024. Effective generation of feasible solutions for integer programming via guided diffusion. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 4107–4118.

A Proof

LEMMA 1 (CONSTRUCTIVE RECOVERY). *If the original ILP (Problem 1) is feasible, then there exists a finite threshold $\mu^* > 0$ such that for any penalty parameter $\mu > \mu^*$, an exact optimal binary solution to Problem 1 can be directly constructed from any optimal solution to Problem 4 without any additional search. This is established by the following two properties:*

- (i) Let $I_C = \{i \in [n] : c_i \neq 0\}$. Suppose $\hat{x}^* \in [0, 1]^n$ is an optimal solution to Problem 5. Then $\hat{x}_i^* \in \{0, 1\}$ for all $i \in I_C$, and the strict feasibility of $\hat{x}^*$ guarantees that every binary

vector $\mathbf{x}^* \in \text{supp } p(\cdot \mid \hat{\mathbf{x}}^*)$ is feasible for Problem 1 with $\mathbf{c}^\top \mathbf{x}^* = \mathbf{c}^\top \hat{\mathbf{x}}^*$. Letting $I_{\hat{\mathbf{x}}^*} = \{i \in [n] : \hat{\mathbf{x}}_i^* \in \{0, 1\}\}$, any $\mathbf{x}^* \in \{0, 1\}^n$ satisfying $\mathbf{x}_i^* = \hat{\mathbf{x}}_i^*$ for all $i \in I_{\hat{\mathbf{x}}^*}$ is therefore an optimal solution to Problem 1.

- (ii) There exists a finite threshold $\mu^* > 0$ such that for any $\mu > \mu^*$, any optimal solution to Problem 4 is also an optimal solution to Problem 5.

PROOF. (i) Let $\hat{\mathbf{x}}^* \in [0, 1]^n$ be an optimal solution to Problem 5. Suppose $\hat{\mathbf{x}}_i^* \in (0, 1)$ for some $i \in I_C$ (so $c_i \neq 0$). We construct a feasible perturbation that strictly decreases the objective. If $c_i > 0$, define $\hat{\mathbf{x}}'$ by $\hat{\mathbf{x}}'_i = 0$ and $\hat{\mathbf{x}}'_j = \hat{\mathbf{x}}_j^*$ for $j \neq i$; if $c_i < 0$, define $\hat{\mathbf{x}}'$ by $\hat{\mathbf{x}}'_i = 1$ and $\hat{\mathbf{x}}'_j = \hat{\mathbf{x}}_j^*$ for $j \neq i$. In either case, $\text{supp } p(\cdot \mid \hat{\mathbf{x}}') \subset \text{supp } p(\cdot \mid \hat{\mathbf{x}}^*)$, so the strict feasibility of $\hat{\mathbf{x}}^*$ is inherited and $\hat{\mathbf{x}}'$ is feasible for Problem 5. By construction $\mathbf{c}^\top \hat{\mathbf{x}}' < \mathbf{c}^\top \hat{\mathbf{x}}^*$, contradicting the optimality of $\hat{\mathbf{x}}^*$. Hence $\hat{\mathbf{x}}_i^* \in \{0, 1\}$ for all $i \in I_C$. For coordinates with $c_i = 0$, no integrality is asserted, and these coordinates do not appear in the objective.

Next, let $\mathbf{x}^* \in \{0, 1\}^n$ satisfy $\mathbf{x}_i^* = \hat{\mathbf{x}}_i^*$ for every $i \in I_{\hat{\mathbf{x}}^*}$, so that $\mathbf{x}^* \in \text{supp } p(\cdot \mid \hat{\mathbf{x}}^*)$. Feasibility of $\hat{\mathbf{x}}^*$ implies

$$\mathbb{E}_{\mathbf{x} \sim p(\cdot \mid \hat{\mathbf{x}}^*)} [g(\mathbf{x})] = 0. \quad (17)$$

Since $g \geq 0$ and the support is non-empty, $g(\mathbf{x}) = 0$ for every $\mathbf{x} \in \text{supp } p(\cdot \mid \hat{\mathbf{x}}^*)$; in particular $\mathbf{A}\mathbf{x}^* \leq \mathbf{b}$, so $\mathbf{x}^*$ is feasible for Problem 1. Using $I_C \subseteq I_{\hat{\mathbf{x}}^*}$,

$$\mathbf{c}^\top \mathbf{x}^* = \sum_{i \in I_C} c_i \mathbf{x}_i^* = \sum_{i \in I_C} c_i \hat{\mathbf{x}}_i^* = \mathbf{c}^\top \hat{\mathbf{x}}^*, \quad (18)$$

and for any feasible $\mathbf{x} \in \{0, 1\}^n$ for Problem 1, optimality of $\hat{\mathbf{x}}^*$ gives $\mathbf{c}^\top \hat{\mathbf{x}}^* \leq \mathbf{c}^\top \mathbf{x}$, so $\mathbf{x}^*$ is optimal for Problem 1.

- (ii) Recall the scalar violation $g(\mathbf{x}) = \sum_j \max(\mathbf{A}_j \mathbf{x} - b_j, 0)$ and its expectation $\hat{g}(\hat{\mathbf{x}}) \triangleq \mathbb{E}_{\mathbf{x} \sim p(\cdot \mid \hat{\mathbf{x}})} [g(\mathbf{x})]$. The domain $\{0, 1\}^n$ is finite, so the set $\{g(\mathbf{x}) : \mathbf{x} \in \{0, 1\}^n, g(\mathbf{x}) > 0\}$ is a finite set of strictly positive reals; let $\rho_{\min} \triangleq \min_{\mathbf{x} \in \{0, 1\}^n, g(\mathbf{x}) > 0} g(\mathbf{x}) > 0$. Set $\mu^* = 2\sqrt{n}\|\mathbf{c}\|_2/\rho_{\min}$ (a finite, instance-specific constant) and assume $\mu > \mu^*$. Let $\hat{\mathbf{x}}^*$ be an optimal solution to the penalized Problem 4 and let $\hat{\mathbf{x}}$ be any feasible solution to Problem 5 (so $\hat{g}(\hat{\mathbf{x}}) = 0$; existence is guaranteed by feasibility of the original ILP). Optimality of $\hat{\mathbf{x}}^*$ implies

$$\mathbf{c}^\top \hat{\mathbf{x}}^* + \mu \hat{g}(\hat{\mathbf{x}}^*) \leq \mathbf{c}^\top \hat{\mathbf{x}} + \mu \hat{g}(\hat{\mathbf{x}}) = \mathbf{c}^\top \hat{\mathbf{x}}. \quad (19)$$

Suppose for contradiction that $\hat{g}(\hat{\mathbf{x}}^*) > 0$. Then some $\bar{\mathbf{x}} \in \text{supp } p(\cdot \mid \hat{\mathbf{x}}^*)$ satisfies $g(\bar{\mathbf{x}}) \geq \rho_{\min}$, and replacing $\hat{\mathbf{x}}^*$ by $\bar{\mathbf{x}}$ in the inequality yields

$$\mu \rho_{\min} \leq \mu g(\bar{\mathbf{x}}) \leq \mathbf{c}^\top \bar{\mathbf{x}} - \mathbf{c}^\top \hat{\mathbf{x}}^* \leq 2\sqrt{n}\|\mathbf{c}\|_2, \quad (20)$$

i.e. $\mu \leq \mu^*$, contradicting $\mu > \mu^*$. Hence $\hat{g}(\hat{\mathbf{x}}^*) = 0$, so $\hat{\mathbf{x}}^*$ is feasible for Problem 5, and the inequality above gives $\mathbf{c}^\top \hat{\mathbf{x}}^* \leq \mathbf{c}^\top \hat{\mathbf{x}}$, establishing optimality. $\square$

THEOREM 2 (SURROGATE MERIT FUNCTION EQUIVALENCE). *The optimization problem defined by the perturbed objective is equivalent to minimizing the surrogate merit function $F_{\mu, \rho}(\hat{\mathbf{x}})$,*

$$\min_{\hat{\mathbf{x}} \in [0, 1]^n} \left\{ F_{\mu, \rho}(\hat{\mathbf{x}}) = (1 - 2\rho)\mathbf{c}^\top \hat{\mathbf{x}} + \mu \mathbb{E}_{\mathbf{x} \sim p(\cdot \mid \hat{\mathbf{x}}), \mathbf{x}' \sim q_\rho(\cdot \mid \mathbf{x})} [g(\mathbf{x}')] \right\}, \quad (7)$$

where $g(\cdot)$ is the aggregate violation defined in Equation 3.

PROOF. The optimization objective after introducing the Stochastic Neighborhood Perturbation (SNP) is defined as:

$$\min_{\hat{\mathbf{x}} \in [0, 1]^n} \mathbb{E}_{\mathbf{x} \sim p(\cdot \mid \hat{\mathbf{x}}), \mathbf{x}' \sim q_\rho(\cdot \mid \mathbf{x})} [\mathbf{c}^\top \mathbf{x}'] + \mu \mathbb{E}_{\mathbf{x} \sim p(\cdot \mid \hat{\mathbf{x}}), \mathbf{x}' \sim q_\rho(\cdot \mid \mathbf{x})} [g(\mathbf{x}')]. \quad (21)$$

Our goal is to show that minimizing this expression is equivalent to minimizing the surrogate merit function $F_{\mu, \rho}(\hat{\mathbf{x}})$ as defined in the theorem. To do this, we only need to analyze the objective term, $\mathbb{E}[\mathbf{c}^\top \mathbf{x}']$, since the penalty term is identical in both expressions.

By the linearity of expectation, we have:

$$\mathbb{E}[\mathbf{c}^\top \mathbf{x}'] = \mathbb{E} \left[\sum_{i=1}^n c_i x'_i \right] = \sum_{i=1}^n c_i \mathbb{E}[x'_i]. \quad (22)$$

The value of each perturbed variable x'_i is given by the SNP operator: $x'_i = (1 - \xi_i)x_i + \xi_i(1 - x_i)$, where $x_i \sim \text{Bernoulli}(\hat{x}_i)$ and $\xi_i \sim \text{Bernoulli}(\rho)$. Since the base sample $\mathbf{x}$ and the perturbation mask ξ are independent, we can compute the expectation of x'_i as follows:

$$\begin{aligned} \mathbb{E}[x'_i] &= \mathbb{E}_{(x_i, \xi_i)} [(1 - \xi_i)x_i + \xi_i(1 - x_i)] \\ &= \mathbb{E}[1 - \xi_i] \mathbb{E}[x_i] + \mathbb{E}[\xi_i] \mathbb{E}[1 - x_i] \quad (\text{by independence}) \\ &= (1 - \mathbb{E}[\xi_i]) \mathbb{E}[x_i] + \mathbb{E}[\xi_i] (1 - \mathbb{E}[x_i]) \\ &= (1 - \rho) \hat{x}_i + \rho(1 - \hat{x}_i) \\ &= \hat{x}_i - \rho \hat{x}_i + \rho - \rho \hat{x}_i \\ &= (1 - 2\rho) \hat{x}_i + \rho. \end{aligned}$$

Substituting this result back into the expression for the expected objective value:

$$\begin{aligned} \mathbb{E}[\mathbf{c}^\top \mathbf{x}'] &= \sum_{i=1}^n c_i ((1 - 2\rho) \hat{x}_i + \rho) \\ &= \sum_{i=1}^n (1 - 2\rho) c_i \hat{x}_i + \sum_{i=1}^n c_i \rho \\ &= (1 - 2\rho) \sum_{i=1}^n c_i \hat{x}_i + \rho \sum_{i=1}^n c_i \\ &= (1 - 2\rho) \mathbf{c}^\top \hat{\mathbf{x}} + \rho \sum_{i=1}^n c_i. \end{aligned}$$

Now, we substitute this simplified expression back into the original optimization problem from Problem 21:

$$\min_{\hat{\mathbf{x}} \in [0, 1]^n} \left\{ \left((1 - 2\rho) \mathbf{c}^\top \hat{\mathbf{x}} + \rho \sum_{i=1}^n c_i \right) + \mu \mathbb{E}_{\mathbf{x} \sim p(\cdot \mid \hat{\mathbf{x}}), \mathbf{x}' \sim q_\rho(\cdot \mid \mathbf{x})} [\max(\mathbf{A}\mathbf{x}' - \mathbf{b}, 0)] \right\}. \quad (23)$$

For a given ILP instance and a fixed perturbation magnitude ρ , the term $\rho \sum_{i=1}^n c_i$ is a constant. In the context of optimization, adding a constant to the objective function does not change the location of its minimum. That is, $\arg \min_{\hat{\mathbf{x}}} (f(\hat{\mathbf{x}}) + C) = \arg \min_{\hat{\mathbf{x}}} f(\hat{\mathbf{x}})$. Therefore, we can drop the constant term $\rho \sum_{i=1}^n c_i$ without affecting the solution.

By dropping this constant, we arrive at the equivalent problem of minimizing the surrogate merit function $F_{\mu, \rho}(\hat{\mathbf{x}})$:

$$\min_{\hat{\mathbf{x}} \in [0, 1]^n} \left\{ (1 - 2\rho) \mathbf{c}^\top \hat{\mathbf{x}} + \mu \mathbb{E}_{\mathbf{x} \sim p(\cdot \mid \hat{\mathbf{x}}), \mathbf{x}' \sim q_\rho(\cdot \mid \mathbf{x})} [g(\mathbf{x}')] \right\}.$$

This concludes the proof. $\square$

THEOREM 3 (DECOMPOSITION OF THE SURROGATE OBJECTIVE). *The SNP-based surrogate merit function $F_{\mu,\rho}(\hat{x})$ from Equation 7 can be expressed as a sum of the scaled original probabilistic objective and a dynamic regularization term $\mathcal{R}_\rho(\hat{x})$:*

$$F_{\mu,\rho}(\hat{x}) = (1 - 2\rho)\mathbf{c}^\top \hat{x} + \mu \mathbb{E}_{\mathbf{x} \sim p(\cdot|\hat{x})}[g(\mathbf{x})] + \mu \mathcal{R}_\rho(\hat{x}), \quad (10)$$

where the regularization term $\mathcal{R}_\rho(\hat{x})$, which represents the expected change in constraint violation induced by the SNP operator, is defined as:

$$\mathcal{R}_\rho(\hat{x}) := \mathbb{E}_{\mathbf{x} \sim p(\cdot|\hat{x})} \left[\mathbb{E}_{\mathbf{x}' \sim q_\rho(\cdot|\mathbf{x})}[g(\mathbf{x}')] - g(\mathbf{x}) \right]. \quad (11)$$

This term effectively measures the robustness of solutions sampled from $p(\cdot|\hat{x})$, penalizing those whose neighbors are likely to have high constraint violations.

PROOF. The proof follows from the law of total expectation. The expectation of the constraint violation in Problem 7 can be decomposed as:

$$\begin{aligned} \mathbb{E}_{\mathbf{x} \sim p, \mathbf{x}' \sim q_\rho}[g(\mathbf{x}')] &= \mathbb{E}_{\mathbf{x} \sim p} \left[\mathbb{E}_{\mathbf{x}' \sim q_\rho(\cdot|\mathbf{x})}[g(\mathbf{x}')] \right] \\ &= \mathbb{E}_{\mathbf{x} \sim p} \left[g(\mathbf{x}) + \left(\mathbb{E}_{\mathbf{x}' \sim q_\rho(\cdot|\mathbf{x})}[g(\mathbf{x}')] - g(\mathbf{x}) \right) \right] \\ &= \mathbb{E}_{\mathbf{x} \sim p}[g(\mathbf{x})] + \mathbb{E}_{\mathbf{x} \sim p} \left[\mathbb{E}_{\mathbf{x}' \sim q_\rho(\cdot|\mathbf{x})}[g(\mathbf{x}')] - g(\mathbf{x}) \right] \\ &= \mathbb{E}_{\mathbf{x} \sim p}[g(\mathbf{x})] + \mathcal{R}_\rho(\hat{x}). \end{aligned}$$

Substituting this back into the definition of $F_{\mu,\rho}(\hat{x})$ in Problem 7 yields the desired result. $\square$

THEOREM 4 (ASYMPTOTIC CONSISTENCY). *As the perturbation magnitude ρ approaches zero, the regularization term vanishes, and the SNP-based objective $F_{\mu,\rho}(\hat{x})$ converges to the original probabilistic objective from Problem 4. Specifically:*

$$\lim_{\rho \rightarrow 0} \mathcal{R}_\rho(\hat{x}) = 0, \quad (12)$$

and consequently,

$$\lim_{\rho \rightarrow 0} F_{\mu,\rho}(\hat{x}) = \mathbf{c}^\top \hat{x} + \mu \mathbb{E}_{\mathbf{x} \sim p(\cdot|\hat{x})}[g(\mathbf{x})]. \quad (13)$$

PROOF. The perturbation distribution $q_\rho(\mathbf{x}' | \mathbf{x})$ is a product of n independent Bernoulli flips, each with parameter ρ . For any fixed pair $\mathbf{x}, \mathbf{x}' \in \{0, 1\}^n$, $q_\rho(\mathbf{x}' | \mathbf{x}) = \rho^d (1 - \rho)^{n-d}$ with $d = \text{Hamming}(\mathbf{x}, \mathbf{x}')$, so as $\rho \rightarrow 0$, $q_\rho(\mathbf{x}' | \mathbf{x}) \rightarrow \mathbb{I}[\mathbf{x}' = \mathbf{x}]$ termwise. Because the sample space $\{0, 1\}^n$ is finite, this termwise convergence yields

$$\lim_{\rho \rightarrow 0} \mathbb{E}_{\mathbf{x}' \sim q_\rho(\cdot|\mathbf{x})}[g(\mathbf{x}')] = \sum_{\mathbf{x}' \in \{0,1\}^n} \lim_{\rho \rightarrow 0} q_\rho(\mathbf{x}' | \mathbf{x}) g(\mathbf{x}') = g(\mathbf{x})$$

by finite-sum convergence (no continuity of g is needed since the domain is discrete). Hence the inner term of the regularizer vanishes: $\lim_{\rho \rightarrow 0} (\mathbb{E}_{\mathbf{x}' \sim q_\rho(\cdot|\mathbf{x})}[g(\mathbf{x}')] - g(\mathbf{x})) = 0$ for every $\mathbf{x}$. Since g is bounded on $\{0, 1\}^n$, dominated convergence gives $\lim_{\rho \rightarrow 0} \mathcal{R}_\rho(\hat{x}) = 0$, and the convergence of $F_{\mu,\rho}(\hat{x})$ follows as $(1 - 2\rho) \rightarrow 1$. $\square$

THEOREM 5 (LINEAR BOUND ON THE REGULARIZATION TERM). *For the ILP formulations in our work, the violation function $g(\mathbf{x})$ is bounded because the decision variables are binary. A concrete upper bound is obtained by summing the worst-case positive violation across*

all m constraints: $M = \sum_{j=1}^m \max\left(0, \sum_{i:A_{ji}>0} A_{ji} - b_j\right)$. Given this, the magnitude of the regularization term $\mathcal{R}_\rho(\hat{x})$ is linearly bounded by the perturbation magnitude ρ . Specifically, the regularization term vanishes at a rate of $O(\rho)$:

$$|\mathcal{R}_\rho(\hat{x})| \leq M\rho. \quad (14)$$

PROOF. Let $r(\mathbf{x}')$ be the marginal distribution of a perturbed sample, defined as $r(\mathbf{x}') = \sum_{\mathbf{x}} p(\mathbf{x} | \hat{x}) q_\rho(\mathbf{x}' | \mathbf{x})$. The exploration term can be rewritten as the difference between the expected violation under the perturbed distribution r and the base distribution p :

$$\mathcal{R}_\rho(\hat{x}) = \mathbb{E}_{\mathbf{x}' \sim r}[g(\mathbf{x}')] - \mathbb{E}_{\mathbf{x} \sim p}[g(\mathbf{x})].$$

The absolute value of this difference is bounded by the maximum value of the function multiplied by the Total Variation (TV) distance between the two distributions:

$$|\mathcal{R}_\rho(\hat{x})| \leq M \cdot \|r - p\|_{TV}.$$

The TV distance is defined as $\|r - p\|_{TV} = \frac{1}{2} \sum_{\mathbf{x}'} |r(\mathbf{x}') - p(\mathbf{x}' | \hat{x})|$. It is upper-bounded by the probability that the perturbed sample $\mathbf{x}'$ is different from the original sample $\mathbf{x}$:

$$\|r - p\|_{TV} \leq \Pr[\mathbf{x}' \neq \mathbf{x}].$$

The event $\mathbf{x}' \neq \mathbf{x}$ occurs if at least one bit is flipped by the SNP operator. The probability that no bits are flipped is $(1 - \rho)^n$. Thus,

$$\Pr[\mathbf{x}' \neq \mathbf{x}] = 1 - (1 - \rho)^n.$$

Using the Bernoulli inequality $(1 - x)^n \geq 1 - nx$ for $x \in (0, 1)$, we have $1 - (1 - \rho)^n \leq 1 - (1 - n\rho) = n\rho$. Combining these inequalities, we get:

$$|\mathcal{R}_\rho(\hat{x})| \leq M \cdot \|r - p\|_{TV} \leq M \cdot \Pr[\mathbf{x}' \neq \mathbf{x}] = M(1 - (1 - \rho)^n) \leq M\rho.$$

This shows that $|\mathcal{R}_\rho(\hat{x})| = O(\rho)$, confirming that the regularizer's influence vanishes linearly as $\rho \rightarrow 0$. $\square$

B Implementation Details

B.1 Feature Design

Following established practices [9, 11, 16], we represent each ILP instance as a weighted bipartite graph $\mathcal{G} = (\mathcal{V} \cup \mathcal{C}, \mathcal{E})$, where $\mathcal{V}$ and $\mathcal{C}$ denote the sets of variable and constraint nodes, respectively. The graph is augmented with feature matrices for variables, constraints, and edges. A detailed description of these features is provided in Table 3.

B.2 GNN Architecture

We adopt the Graph Convolutional Network (GCN) architecture proposed by Gasse et al. [9]. This design leverages a bipartite graph representation of the ILP, which effectively captures the problem's relational structure and facilitates generalization across diverse problem instances.

Bipartite Graph Representation. An ILP instance is encoded as a bipartite graph $\mathcal{G} = (\mathcal{V}, \mathcal{C}, \mathcal{E})$, where $\mathcal{V}$ is the set of variable nodes, $\mathcal{C}$ is the set of constraint nodes, and $\mathcal{E}$ contains edges connecting variables to the constraints in which they participate. Nodes and edges are augmented with feature vectors describing their respective properties (e.g., objective coefficients for variables, RHS values for constraints), as detailed in Table 3.

Table 3: Description of variable, constraint, and edge features in our bipartite graph representation.

Tensor	Feature	Description
Variable Node Features	Objective Coefficient	The normalized coefficient of the variable in the objective function.
	Average Coefficient	The average coefficient of the variable across all constraints it appears in.
	Degree	The degree of the variable node in the bipartite graph.
	Max Coefficient	The maximum coefficient of the variable across all constraints.
	Min Coefficient	The minimum coefficient of the variable across all constraints.
Constraint Node Features	Average Coefficient	The average of all non-zero coefficients in the constraint.
	Degree	The degree of the constraint node in the bipartite graph.
	Bias	The normalized right-hand-side (RHS) value of the constraint.
Edge Features	Coefficient	The coefficient of the variable in the corresponding constraint.

GCN Architecture. The GCN processes this graph through a series of convolutional layers to produce an embedding for each variable, which is then mapped to a probability. The architecture comprises a graph convolution block, an output layer, and a pre-normalization layer to ensure training stability.

Graph Convolution Layer. Given the bipartite structure, a single graph convolution operation involves two message-passing phases:

- (i) Variable-to-Constraint Pass: Each constraint node c_i aggregates feature vectors from its neighboring variable nodes v_j :

$$\mathbf{h}'_{c_i} \leftarrow f_C \left(\mathbf{h}_{c_i}, \sum_{(j,i) \in \mathcal{E}} g_C(\mathbf{h}_{c_i}, \mathbf{h}_{v_j}, \mathbf{e}_{ji}) \right) \quad (24)$$

- (ii) Constraint-to-Variable Pass: Each variable node v_j then aggregates information from its updated neighboring constraint nodes c_i :

$$\mathbf{h}'_{v_j} \leftarrow f_V \left(\mathbf{h}_{v_j}, \sum_{(j,i) \in \mathcal{E}} g_V(\mathbf{h}'_{c_i}, \mathbf{h}_{v_j}, \mathbf{e}_{ji}) \right) \quad (25)$$

Here, $\mathbf{h}$ denotes node feature vectors, and the functions f_C, f_V, g_C, g_V are implemented as 2-layer Multi-Layer Perceptrons (MLPs) with ReLU activations. This two-phase process is repeated for a specified number of steps, defining the GCN’s depth.

Output Layer. After the final convolution, the updated variable node embeddings $\mathbf{h}'_{v_j}$ are passed through a final 2-layer MLP to produce logits. These logits are then transformed via a sigmoid function into probabilities $\hat{\mathbf{x}} \in^n$, which parameterize the multivariate Bernoulli distribution for sampling solutions.

Pre-normalization Layer. To stabilize training with the un-normalized sum aggregations in Equations 24 and 25, a pre-normalization step is applied to the aggregated messages. This simple affine transformation, $\mathbf{z}' \leftarrow (\mathbf{z} - \beta)/\sigma$, uses a mean β and standard deviation σ pre-computed on the training set. This technique preserves the model’s ability to reason about cardinalities while preventing the explosion of activation values.

B.3 Solver Integration via Local Branching

The high-quality heuristic solutions produced by our trained model are used to accelerate traditional solvers, such as Gurobi and SCIP. We employ a simplified variant of the local branching strategy [8], which integrates our model’s predictions in two key ways:

- (1) Warm Start (MIP Start): The predicted solution $\mathbf{x}$ is supplied to the solver as an initial feasible solution. This provides the solver with a strong incumbent from the outset, enabling significant pruning of the search tree and accelerating convergence to the optimal solution.
- (2) Local Branching Constraint: To guide the solver’s search toward promising regions of the solution space, we introduce a constraint that defines a trust region around our predicted solution $\hat{\mathbf{x}}$. This constraint limits the search to solutions within a specified Hamming distance k :

$$\sum_{i:\hat{x}_i=1} (1 - x_i) + \sum_{i:\hat{x}_i=0} x_i \leq \Delta \quad (26)$$

This constraint ensures that any new solution $\mathbf{x}$ found by the solver can differ from our model’s prediction $\hat{\mathbf{x}}$ in at most Δ variable assignments, effectively focusing the search on a high-potential neighborhood.

This hybrid strategy is straightforward to implement with modern solvers. In Gurobi, the warm start is provided by setting the ‘Start’ attribute of the variables, and the local branching constraint is added to the model via standard API calls. A similar procedure is followed in SCIP, using `SCIPcreateSol` and `SCIPaddSol` for the warm start and `SCIPaddCons` to add the local branching constraint.

B.4 Hyperparameter Settings

The hyperparameters used for training PUMA are detailed in Table 4. These settings were kept consistent across benchmarks where feasible, with targeted adjustments to the learning rate, GNN depth, and penalty parameters to accommodate the unique structural properties of each problem class.

For inference, both our method and the DiffILO baseline employ the adaptive local branching strategy described in Appendix B.3. We set the local branching neighborhood size (Δ) to 400, 400, and 200 for the CA, IS, and SC problems, respectively. For the Primal-Search (PS) methods and the PS-based ConPaS baseline, we adopted

Table 4: Hyperparameters for training PUMA.

Hyperparameter	SC	IS	CA	Description
Embedding Size	32	32	32	The embedding dimension in the GNN.
GNN Depth	3	10	10	The number of graph convolution layers.
Batch Size	5	5	5	Number of ILP instances per training batch.
Num Base Samples (K)	15	15	15	Number of base solutions sampled for the expectation.
Num Perturbed Samples (M)	10	10	10	Number of perturbed solutions sampled per base solution.
Epochs	1,200	1,200	2,400	Maximum number of training epochs.
Optimizer	Adam	Adam	Adam	Optimizer used for training.
Learning Rate	1e-4	8e-5	8e-5	Initial learning rate for the Adam optimizer.
LR Annealing Period (T_{lr})	200	200	200	Period for cosine annealing of the learning rate.
Initial Perturbation (ρ_0)	0.02	0.02	0.02	Initial perturbation magnitudes for the SNP operator.
Initial Penalty (μ_0)	5.0	100.0	15.0	Initial value of the penalty coefficient μ .
Penalty Step ($\Delta\mu$)	0.01	1.0	0.001	Step size for linearly increasing μ .

the hyperparameters specified in their original publications, as summarized in Table 5.

Table 5: Search hyperparameters (k_0, k_1, Δ) for the PS and ConPaS baselines.

Dataset	Hyperparameters (k_0, k_1, Δ)
CA	(800, 0, 10)
IS	(400, 400, 25)
SC	(600, 0, 10)

C Benchmark Details

C.1 Main Experiments

We evaluate the performance of our method on three widely recognized Integer Linear Programming (ILP) benchmarks: Combinatorial Auction (CA), Maximum Independent Set (IS), and Set Cover (SC). These datasets are standard in the literature and represent a diverse array of challenging combinatorial optimization problems. We utilize instances of varying scales to assess our method’s efficacy in both main performance benchmarks and generalization experiments.

Combinatorial Auction (CA). The Combinatorial Auction problem involves allocating a set of indivisible items to a group of bidders, where bidders submit bids on specific bundles of items. The objective is to select a subset of non-conflicting bids to maximize total revenue, ensuring that each item is sold at most once. Let $\mathcal{B}$ denote the set of all bids. For each bid $j \in \mathcal{B}$, let v_j be its value and S_j be the requested bundle of items. The ILP formulation is given by:

$$\begin{aligned}
& \max_{\mathbf{x}} \quad \sum_{j \in \mathcal{B}} v_j x_j \\
& \text{s.t.} \quad \sum_{j: i \in S_j} x_j \leq 1, \quad \forall i \in \text{Items} \\
& \quad \quad x_j \in \{0, 1\}, \quad \forall j \in \mathcal{B}
\end{aligned} \tag{27}$$

where x_j is a binary indicator variable that equals 1 if bid j is accepted, and 0 otherwise.

Maximum Independent Set (IS). The Maximum Independent Set problem aims to identify the largest subset of vertices in a graph such that no two vertices in the subset are adjacent. Given a graph $G = (V, E)$, the problem is formulated as follows:

$$\begin{aligned}
& \max_{\mathbf{x}} \quad \sum_{i \in V} x_i \\
& \text{s.t.} \quad x_i + x_j \leq 1, \quad \forall (i, j) \in E \\
& \quad \quad x_i \in \{0, 1\}, \quad \forall i \in V
\end{aligned} \tag{28}$$

where x_i is a binary variable set to 1 if vertex i is included in the independent set, and 0 otherwise.

Set Cover (SC). The Set Cover problem addresses the challenge of covering a universe of elements using the minimum number of subsets from a given family. Given a universe $\mathcal{U}$ and a collection of subsets $\mathcal{S} = \{S_1, S_2, \dots, S_n\}$, the formulation is:

$$\begin{aligned}
& \min_{\mathbf{x}} \quad \sum_{j=1}^n x_j \\
& \text{s.t.} \quad \sum_{j: e_i \in S_j} x_j \geq 1, \quad \forall e_i \in \mathcal{U} \\
& \quad \quad x_j \in \{0, 1\}, \quad \forall j \in \{1, \dots, n\}
\end{aligned} \tag{29}$$

where x_j is a binary variable equal to 1 if set S_j is selected, and 0 otherwise.

Table 6 summarizes the characteristics and maximum sizes of the instances used in our main experiments.

Table 6: Maximum problem sizes for main experiments.

Dataset	Constraint Number	Number of Binary Variables
CA	974	2,500
IS	9,929	2,000
SC	3,000	2,000

C.2 Details of Generalization Experiments

Large-scale Synthetic Benchmarks. The parameters for the large-scale synthetic benchmarks, utilized in Section 5.4, are detailed

below. Each benchmark is scaled to approximately double the size of the original instances:

- Combinatorial Auction (CA): Generated with 1,000 items and 5,000 bids.
- Independent Set (IS): Constructed on graphs with 4,000 nodes and an affinity parameter of 5.
- Set Cover (SC): Created using matrices with 5,000 rows and 4,000 columns.

The maximum problem sizes for these generalization experiments are presented in Table 7.

Table 7: Maximum problem sizes for generalization experiments.

Dataset	Constraint Number	Number of Binary Variables
CA	1,951	5,000
IS	199,006	4,000
SC	5,000	4,000

MIPLIB Dataset Details. To assess performance in real-world scenarios, we utilize two distinct subsets from the MIPLIB 2017 library. First, following Wang et al. [42], we employ the Capacitated Vertex Separator (CVS) subset. We use instances cvs16r139–94, cvs16r70–62, and cvs16r89–60 for training PUMA, while cvs16r106–72 and cvs16r128–89 are reserved for testing. Second, we curated a “MINE” subset consisting of Multi-period Mine Production Scheduling instances. The training set includes reblock420, reblock166, mine–166–5, and mine–90–10, with reblock115 and reblock354 serving as the test set. The results of these experiments are discussed in Appendix D.5.

D Additional Experimental Results

D.1 Analysis of Prediction Accuracy

Beyond objective values, we analyze the structural quality of the predicted solutions. We measure accuracy by treating the problem as a binary classification task, where the goal is to predict the variable assignments of the Best-Known Solution (BKS). A high accuracy indicates that a predicted solution is not only high-quality but also structurally similar to the BKS, making it a more effective starting point for a solver. As shown in Figure 7(a) and quantified in Table 8, PUMA achieves superior overall accuracy across all benchmarks, demonstrating its advanced predictive capabilities.

To conduct a more granular analysis, we evaluate the prediction accuracy on two distinct subsets of variables: “negative samples” (variables with a value of 0 in the BKS) and “positive samples” (variables with a value of 1 in the BKS). This detailed breakdown, presented in Figure 7(b-c) and Table 9, provides deeper insight into each model’s behavior. PUMA consistently achieves a more balanced accuracy across both subsets compared to the baselines. For instance, in the Combinatorial Auction (CA) benchmark, it achieves a 5.27 percentage point improvement on negative samples over the next-best method, indicating a significantly reduced rate of false positives. This balanced accuracy is crucial, as it confirms that solutions from PUMA are structurally aligned with the optimal

solution, rendering them more effective for solver initialization and guidance.

Table 8: Average overall prediction accuracy of different approaches across 100 test instances. The best result is marked in bold. “Gurobi” represents Gurobi’s built-in heuristic and “DiffILO-L2” is a variant of DiffILO with L2 regularization.

Method	CA	IS	SC
Gurobi	89.02%	54.15%	95.73%
DiffILO	91.30%	85.76%	98.68%
DiffILO-L2	91.60%	84.68%	98.68%
PUMA	93.04%	88.38%	98.72%
Improvement	1.44 p.p.	2.62 p.p.	0.04 p.p.

Table 9: Average prediction accuracy on negative and positive sample subsets across 100 test instances. The best value in each category is marked in bold. ‘Gurobi’ refers to Gurobi’s built-in heuristic, and ‘DiffILO-L2’ is the variant with L2 regularization.

Accuracy on Negative Samples (BKS=0)			
Method	CA	IS	SC
Gurobi	26.89%	24.53%	2.41%
DiffILO	41.51%	80.79%	90.24%
DiffILO-L2	48.66%	78.25%	90.48%
PUMA	53.93%	85.21%	88.10%
Improvement	5.27 p.p.	4.42 p.p.	-2.38 p.p.

Accuracy on Positive Samples (BKS=1)			
Method	CA	IS	SC
Gurobi	94.19%	75.44%	97.70%
DiffILO	95.50%	89.38%	98.88%
DiffILO-L2	95.19%	89.44%	98.83%
PUMA	96.22%	90.59%	98.93%
Improvement	0.72 p.p.	1.15 p.p.	0.05 p.p.

D.2 Training Convergence measured by Validation Performance

To demonstrate PUMA’s capability to escape suboptimal local minima and sustain optimization, we compare its training dynamics against the state-of-the-art unsupervised method. During training, we periodically sample 1,000 solutions from the model’s current output distribution and record the best objective value found on a validation set. This metric serves as a proxy for observing the quality of solutions the model is learning to generate over time.

Figure 8 illustrates the evolution of the validation objective. The baseline quickly plateaus, reflecting the diversity loss and local optima entrapment discussed in Section 1. In contrast, PUMA maintains

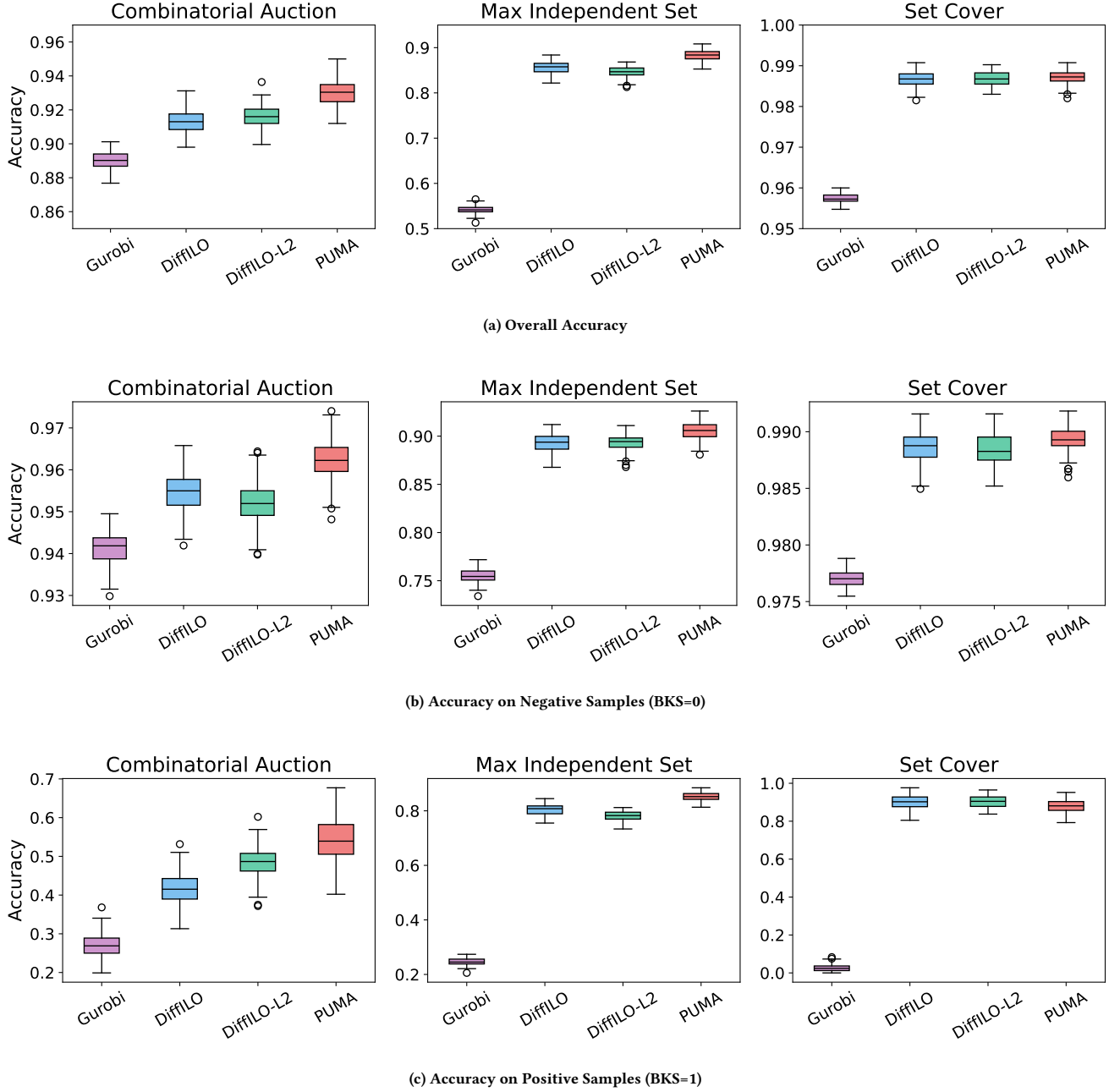


Figure 7: Comparison of average prediction accuracy against the Best-Known Solution (BKS) across 100 test instances. (a) shows the overall accuracy, where our method, PUMA, comprehensively outperforms all baselines. (b) and (c) detail the accuracy on the subsets of “negative samples” (variables with a value of 0 in the BKS) and “positive samples” (variables with a value of 1 in the BKS), respectively.

a trajectory of continuous improvement, verifying the effectiveness of our SNP operator. By maintaining solution diversity, SNP enables broad early-stage exploration that transitions into fine-grained refinement, effectively escaping suboptimal local minima.

D.3 Extended Evaluation on Improving Traditional Solvers

In this section, we provide additional experimental results to comprehensively evaluate the capability of PUMA. We first report the

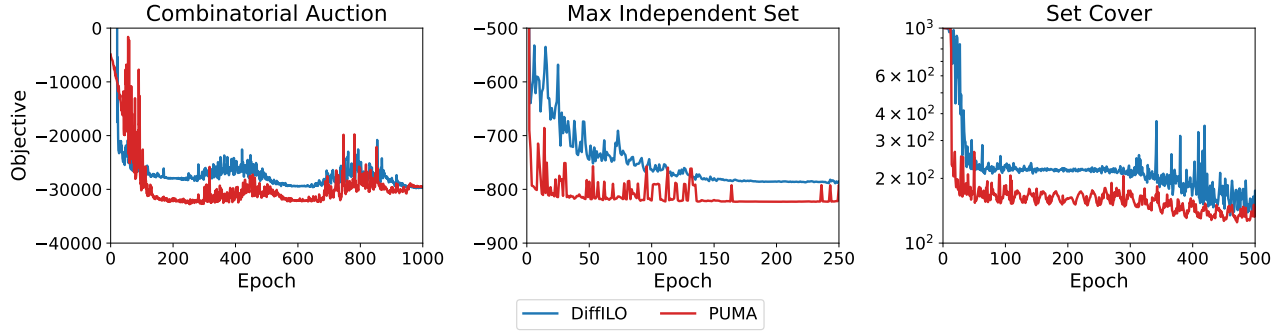


Figure 8: Best objective value found on the validation set as a function of training epochs. The y-axis represents the best objective from 1,000 sampled solutions. PUMA consistently finds superior solutions, demonstrating its ability to escape poor local optima that may trap baseline methods.

detailed performance metrics specifically for the open-source solver SCIP. Subsequently, we introduce the Primal Integral (PI) metric to assess the convergence speed and stability across both Gurobi and SCIP solvers.

Detailed Performance on Improving SCIP Solver. To demonstrate the general applicability of our framework, we extend our evaluation to SCIP, a leading open-source constraint integer programming solver. We integrate PUMA with SCIP using the same local branching strategy described in Section B.3 and compare its performance against SCIP alone and SCIP enhanced with other learning-based methods (PS and DiffiLO).

Figure 9 visualizes the performance over time, plotting the primal gap. The results clearly show that PUMA +SCIP consistently finds higher-quality solutions more rapidly than all other configurations. Table 10 quantifies this advantage, reporting the average objective value and absolute primal gap (Gap_{abs}) after 1,000 seconds. Our method achieves a substantial reduction in the final primal gap across all benchmarks, with improvements of 58.43% for CA, 49.25% for IS, and 66.67% for SC over the default SCIP method. These results confirm that PUMA is not tailored to a specific solver but serves as a versatile and powerful accelerator for state-of-the-art ILP solvers.

Primal Integral Analysis. To comprehensively assess the solver’s performance throughout the entire time limit, we employ the Primal Integral (PI) metric. Defined as $I(t) = \int_0^t \gamma_{\text{rel}}(\tau) d\tau$, the PI represents the cumulative area under the relative primal gap curve. Unlike point-in-time metrics (like the final gap), the PI captures both the speed of convergence and the quality of the solution; a smaller value indicates that a method finds high-quality solutions earlier and maintains them.

Figure 10 presents the PI results for both Gurobi and SCIP. PUMA consistently achieves the lowest integral across both solvers, demonstrating superior solution quality and convergence speed compared to baselines.

D.4 Additional Results on Generalization to Large-scale Benchmarks

This section presents detailed quantitative results for the generalization experiments outlined in Section 5.4. Table 11 summarizes the specific objective values and primal gaps across three benchmarks.

PUMA demonstrates superior generalization capabilities, consistently achieving the best performance across all tasks. As shown in the “Improvement” row, PUMA significantly reduces the absolute primal gap compared to the Gurobi solver. Most notably, on the CA benchmark, our method reduces the primal gap to a negligible 0.12%, marking a substantial improvement over Neural Diving (12.34%) and confirming the model’s robustness on large-scale instances.

Extreme large-scale evaluation on IS. To probe PUMA’s scalability further, we evaluate it on IS instances with 10,000 variables each under a 1,000-second time limit, well beyond the size used for training. Table 12 reports the mean objective and standard deviation: PUMA +Gurobi improves the objective by more than 6% over standalone Gurobi while exhibiting an order of magnitude lower variance, evidence that the learned distribution scales gracefully and remains robust at instance sizes never seen during training.

Table 12: Extreme large-scale IS evaluation ($n = 10,000$ variables, 1,000-second time limit). Mean $\pm$ std over the test set.

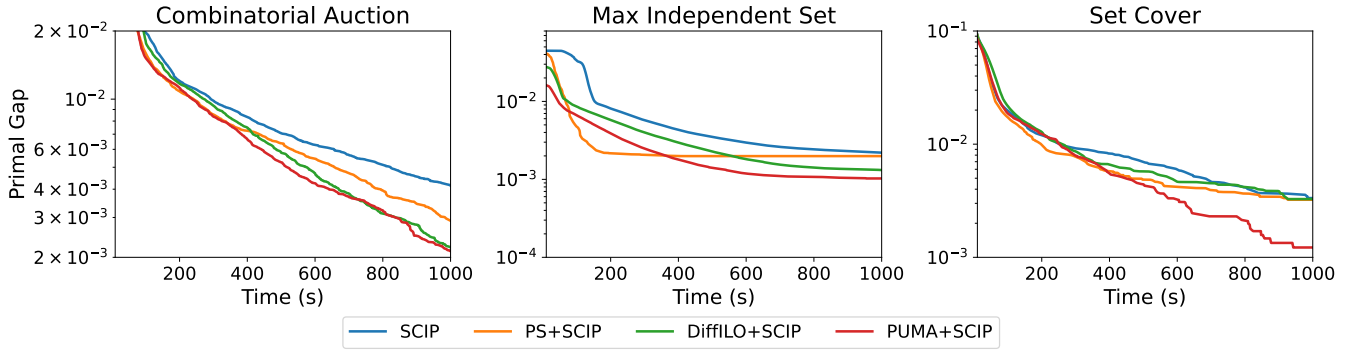
Method	Obj $\uparrow$ (mean $\pm$ std)
Gurobi	3950.10 $\pm$ 109.04
PUMA +Gurobi	4189.97 $\pm$ 19.79

D.5 Additional Results on MIPLIB Datasets

We further evaluate the practical efficacy of PUMA on three real-world subsets drawn from MIPLIB 2017 [12]. The first is the capacitated vertex separator (CVS) subset [42]. The second is a “MINE” subset for multi-period mine production scheduling. The third is a “neos” subset constructed following the DiffiLO protocol [11], containing all instances whose names start with neos and whose mps files have fewer than 500,000 lines (25 instances total, split 20/5

Table 10: Average objective and absolute primal gap (Gap_{abs}) after 1,000 seconds using the SCIP solver over 100 test instances. Best results are in bold. The Best-Known Solution (BKS) for each problem is listed in the header.

Method	CA (BKS: 37011.96)		IS (BKS: 837.67)		SC (BKS: 86.47)	
	Obj $\uparrow$	Gap $_{\text{abs}}\downarrow$	Obj $\uparrow$	Gap $_{\text{abs}}\downarrow$	Obj $\downarrow$	Gap $_{\text{abs}}\downarrow$
SCIP	36983.96	128.00	835.68	1.99	86.77	0.30
PS+SCIP	36930.29	81.67	835.86	1.81	86.76	0.29
DiffILO+SCIP	36955.18	56.78	836.42	1.25	86.77	0.30
PUMA +SCIP	36958.75	53.21	836.66	1.01	86.57	0.10
Improvement		58.43%		49.25%		66.67%

**Figure 9: Primal Gap over time. Performance comparison of SCIP, PS+SCIP, DiffILO+SCIP, and PUMA +SCIP across the three benchmarks. Lower values are better. PUMA consistently enables the solver to find better solutions faster.****Table 11: Average objective and absolute primal gap (Gap) after 1,000 seconds on 100 large-scale test instances. Best results are in bold. The arrows ($\uparrow$, $\downarrow$) indicate whether higher or lower values are better. The “Improvement” row shows the percentage reduction in Gap_{abs} achieved by PUMA +Gurobi over the standalone Gurobi solver.**

Method	CA (BKS: 73896)		IS (BKS: 1679)		SC (BKS: 169)	
	Obj $\uparrow$	Gap $\downarrow$	Obj $\uparrow$	Gap $\downarrow$	Obj $\downarrow$	Gap $\downarrow$
Gurobi	73762	133	1658	22	170.1	0.8
DiffILO+Gurobi	72663	1233	1665	14	170.1	0.8
PUMA +Gurobi	73875	21	1679	0.0	170.0	0.7
Improvement		84.6%		100%		12.8%

for train/test). Together these subsets cover three distinct problem families rather than a handful of arbitrarily chosen instances, providing family-wise structural coverage of MIPLIB.

Table 13 reports objective values across representative instances from all three subsets, including large neos instances of up to $\approx 40k$ variables. PUMA +Gurobi matches or improves over standalone Gurobi on every instance. Crucially, on the more challenging instances such as cvs16r128-89 and reblock354, where Gurobi and

DiffILO stagnate at suboptimal solutions, PUMA successfully identifies better objectives (e.g., improving from -96.00 to -97.00 on CVS), confirming that the learned distribution scales well beyond the typical training-size regime and helps the solver escape local optima.

Table 13: MIPLIB 2017 results on representative instances from the CVS, MINE, and neos subsets. PUMA +Gurobi consistently matches or improves over standalone Gurobi, including on the largest neos instance ($\approx 40k$ variables).

Subset	Instance	Gurobi	PUMA +Gurobi
CVS	cvs16r106-72	-81.00	-81.00
	cvs16r128-89	-96.00	-97.00
MINE	reblock115	-36,800,603	-36,800,603
	reblock354	-39,280,200	-39,280,400
neos	neos_829552	6.38	5.71
	neos_831188	45.21	43.97
	neos_18	0.26	0.14

D.6 Comparison with Additional Baselines

To situate PUMA within the broader landscape of learning-based ILP methods, we benchmark it against ConPaS [19], a representative supervised contrastive learning framework. Since the official code

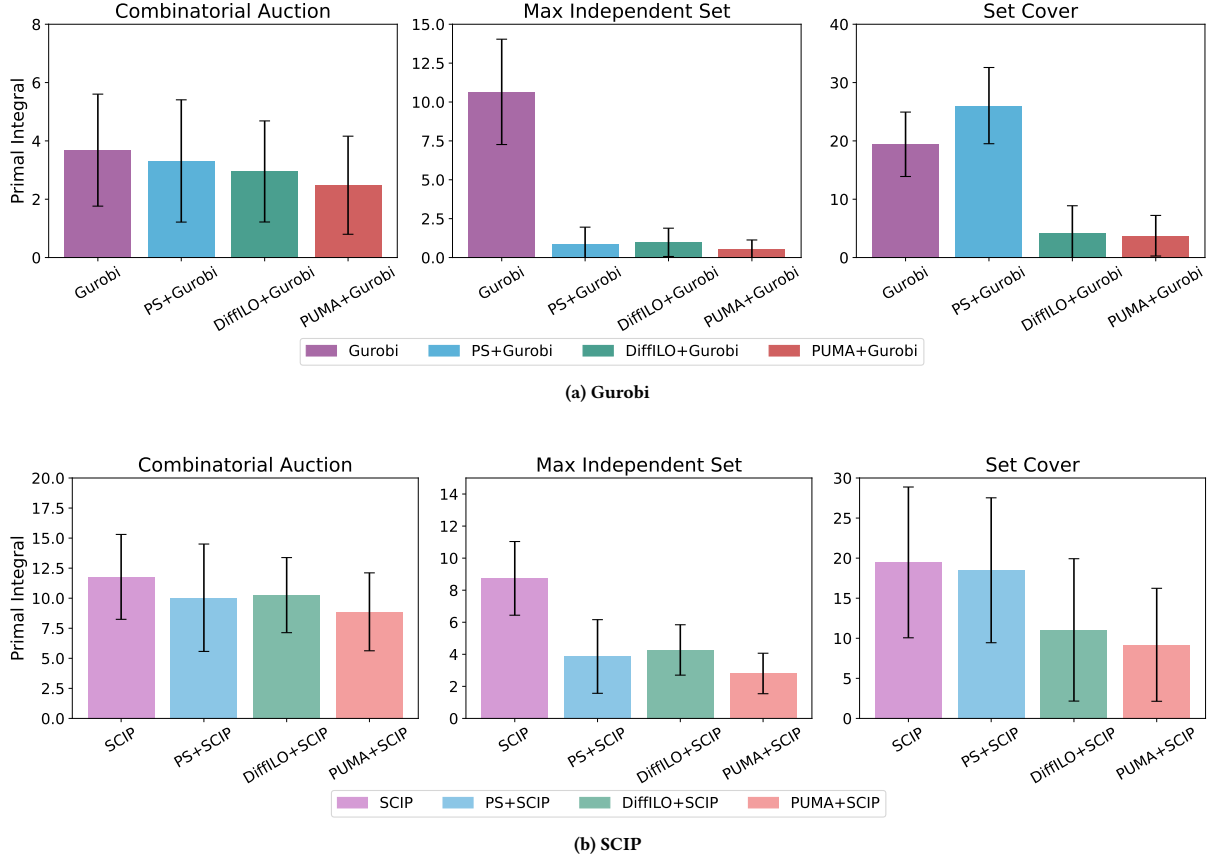


Figure 10: Average primal integral (area under the primal gap curve) over 100 test instances for (a) Gurobi and (b) SCIP. A lower value indicates better overall performance. PUMA consistently achieves the lowest integral, demonstrating superior solution quality and convergence speed across both solvers.

is unavailable, we implemented a faithful reproduction based on the paper’s details. Figure 11 presents the primal gap evolution, comparing PUMA +Gurobi against our reproduced ConPaS+Gurobi and other baselines. The results demonstrate that PUMA consistently outperforms the supervised counterpart. This finding is significant as it confirms that our unsupervised paradigm can surpass advanced supervised methods without the prohibitively expensive overhead of generating labeled training data.

Furthermore, we distinguish our approach, which predicts initial solutions to enhance traditional solvers, from end-to-end neural solvers like DIMES. DIMES learns policies via probabilistic reformulation but treats the task purely as a graph decision problem. To directly compare these paradigms we evaluate PUMA and DIMES in two settings: a subset of the SATLIB benchmark (commonly used to evaluate DIMES), and our primary IS benchmark (instances with 2,000 nodes).

SATLIB. Table 14 reports the objective and total wall-clock time over the SATLIB test set, including the full DIMES family (DIMES alone and DIMES+Gurobi) as additional baselines. Pure PUMA already attains an objective on par with DIMES while being roughly

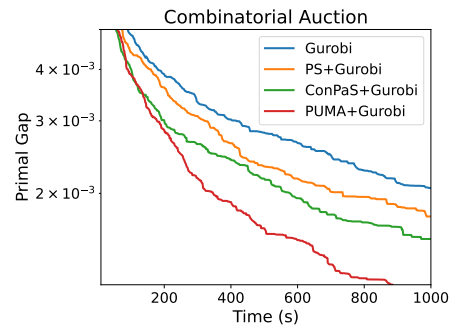


Figure 11: Performance comparison of the average primal gap on CA (100 test instances). PUMA +Gurobi is compared against Gurobi, PS+Gurobi [16], and our reproduction of ConPaS [19].

6× faster in total runtime. PUMA +Gurobi then matches the Gurobi optimum and does so almost twice as fast as standalone Gurobi,

and substantially faster than DIMES+Gurobi which inherits the extra overhead of cascaded inference.

Table 14: Comparison with the DIMES family on the SATLIB benchmark. “Time” is the total wall-clock time over the test set (minutes). PUMA alone is competitive with DIMES at a fraction of the cost, and PUMA +Gurobi matches Gurobi’s optimum substantially faster than both Gurobi and DIMES+Gurobi.

Method	Obj $\uparrow$	Time (min) $\downarrow$
Gurobi	425.96	38.97
DIMES	423.28	30.44
DIMES+Gurobi	424.93	60.74
PUMA	423.10	4.80
PUMA +Gurobi	425.96	24.20

IS (2,000 nodes). Table 15 reports the analogous comparison on our primary IS benchmark, where the time column is mean wall-clock per instance (seconds). Pure PUMA already dominates DIMES on both solution quality (822.2 vs. 817.3) and per-instance runtime (2.16 s vs. 17.71 s), with an $8\times$ speedup at a slightly higher objective. This confirms that PUMA outperforms DIMES on our main problem setting in both solution quality and computational efficiency.

Table 15: Comparison with DIMES on the IS benchmark (2,000 nodes). “Time” is mean wall-clock per instance (seconds).

Method	Obj $\uparrow$	Time (s/instance) $\downarrow$
DIMES	817.3	17.71
PUMA	822.2	2.16

D.7 Ablation Studies

We conduct a series of ablation studies to analyze the contributions of the key components and hyperparameters of our proposed framework.

Effectiveness of the SNP Operator. To isolate the contribution of our proposed SNP operator, we conducted an experiment to verify that its benefits extend beyond merely increasing the number of samples. We compare PUMA against a variant of DiffiLO that is configured to use an identical total number of Monte Carlo samples ($K \times M$) for its loss calculation, but without applying the perturbation step. As illustrated in Figure 12, PUMA maintains a significant performance advantage over this equally-sampled DiffiLO baseline. This result confirms that the structured exploration induced by the SNP operator is the critical factor driving our model’s superior performance, rather than simply a larger sampling budget.

Sensitivity to the Number of Perturbed Samples. We assess the robustness of PUMA by analyzing its sensitivity to the number of perturbed samples, M . Figure 13 examines the effect of varying the

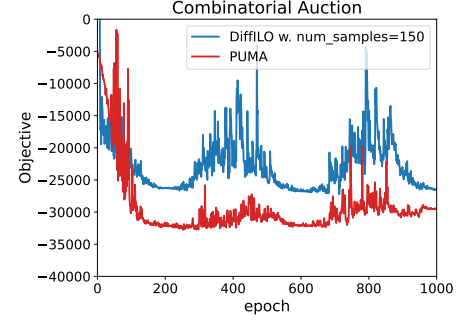


Figure 12: Validation objective comparison between PUMA and DiffiLO configured with an equivalent number of samples ($K \times M$). The results show that the structured exploration from the SNP operator, not just the sample count, is the key driver of PUMA’s superior performance.

number of perturbed samples, M , on the IS benchmark. Performance improves as M increases from 5 but quickly plateaus. Our chosen value of $M = 10$ offers a favorable trade-off between computational cost and performance, yet even smaller values yield strong results. This demonstrates the sample efficiency of the SNP operator in guiding the learning process.

Training-time overhead of perturbation. A natural concern is whether injecting M perturbed samples per base sample inflates the per-epoch training cost. Table 16 reports the wall-clock per-epoch training time on the CA benchmark (RTX 4070, batch size 5, 64 batches/epoch) for several combinations of the base-sample count K and perturbation count M , where $M = 0$ recovers the DiffiLO baseline. Two observations stand out: (i) the per-epoch cost is dominated by the base-sample count K ; increasing M from 0 to 8 adds only a small overhead because the M perturbed samples share the $Ax - b$ computation within a single batched matrix multiplication; and (ii) although PUMA introduces a 7–25% per-epoch overhead at $M = 8$, the perturbation prevents premature collapse and yields convergence in far fewer epochs, so the total wall-clock time to reach a comparable solution quality is on par with or shorter than that of DiffiLO.

Table 16: Per-epoch wall-clock training time (seconds) on the CA benchmark for different combinations of base-sample count K and perturbation count M . $M = 0$ corresponds to the DiffiLO baseline. Cost is dominated by K ; the relative overhead from increasing M is small.

$K \backslash M$	$M = 0$	$M = 2$	$M = 8$
8	14.81	15.74	15.27
32	22.84	27.08	27.63
64	36.58	43.50	43.62

Robustness of the Initial Penalty μ_{init} . A common practical question is how sensitive PUMA is to the initial value of the constraint penalty μ . We adopt an adaptive update $\mu_{k+1} = \mu_k + \eta \cdot (\text{viol} - \tau)$

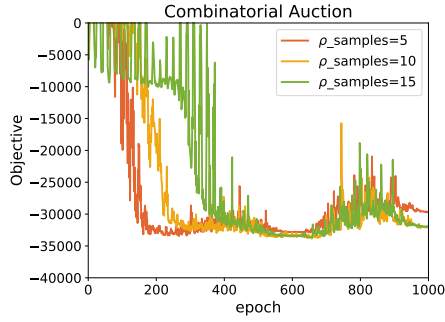


Figure 13: Ablation study on the CA benchmark, showing the impact of the number of perturbed samples M on the validation objective during training.

at every epoch so that μ self-adjusts during training; the initial value μ_{init} is then the only hyperparameter that must be supplied, and a simple closed-form heuristic $\mu_{\text{init}} \approx \sqrt{2m}/\|A\|_F$ provides a problem-aware default. To verify that the adaptive schedule absorbs reasonable deviations from this heuristic, we vary μ_{init} across two orders of magnitude on the IS benchmark (Table 17). The objective spread is below 0.33 across the entire range, and every configuration substantially outperforms the DiffILO baseline (-803.27), showing that PUMA is highly robust to the initial penalty choice and does not require benchmark-specific tuning.

Table 17: Robustness of PUMA to the initial penalty parameter μ_{init} on the IS benchmark. Objective spread is below 0.33 across two orders of magnitude. The DiffILO baseline objective is -803.27 for reference.

μ_{init}	1	10	100	200
Obj $\uparrow$	-825.55	-827.25	-828.23	-825.67

Feasibility Ratio of Sampled Solutions. While the absolute objective of the best feasible sample is the metric that ultimately drives ILP performance, the raw feasibility ratio of the learned distribution is a useful diagnostic for the constraint-handling behavior of unsupervised methods. Following the protocol of Section 5.1 (1,000 i.i.d. Bernoulli samples per instance at test time), we report the mean feasibility ratio (with std across instances) for the PS, DiffILO, and PUMA distributions in Table 18. On the tightly-constrained IS and SC benchmarks PUMA produces substantially more feasible samples than DiffILO. PS collapses to near-zero feasibility because it only aligns its output with the distribution of Gurobi-collected solutions without modeling constraint feasibility during training. Combined with the dominant feasible-objective metric in Table 1, this confirms that PUMA’s gains stem from a constraint-aware distribution rather than an artifact of sampling.

Impact of GNN Backbone. We investigate whether the choice of the graph neural network architecture affects the final solver performance. Table 19 compares the performance on the CA benchmark using Graph Attention Networks (GAT) and Graph Convolutional

Table 18: Mean feasibility ratio (std across instances) of 1,000 i.i.d. Bernoulli samples drawn from each method’s learned distribution.

Method	CA	IS	SC
PS	$0.1\% \pm 1.0\%$	$0.0\% \pm 0.0\%$	$0.0\% \pm 0.0\%$
DiffILO	$99.2\% \pm 5.6\%$	$70.8\% \pm 28.0\%$	$33.1\% \pm 25.3\%$
PUMA	$85.1\% \pm 14.5\%$	$93.5\% \pm 9.7\%$	$64.4\% \pm 10.5\%$

Networks (GCN) as the encoder backbone. While both architectures allow PUMA to outperform the Gurobi baseline (36968.23), the GCN-based implementation achieves a slightly higher objective (37000.38 vs. 36992.72). This suggests that while our framework is robust to the choice of GNN, capturing global structural information (via GCN’s aggregation) may offer marginal benefits for these combinatorial tasks.

Table 19: Performance comparison with different GNN backbones on the CA benchmark. Both variants outperform the baseline, with GCN showing a slight advantage.

Method	Obj $\uparrow$
Gurobi	36968.23
PUMA (GAT)+Gurobi	36992.72
PUMA (GCN)+Gurobi	37000.38

D.8 Comparison with Naïve Initialization Strategies

To validate that the performance gains of PUMA stem from learned structural insights rather than from simple randomization, we compare PUMA against naïve initialization strategies, including large-scale random sampling and LP-rounding. For each random baseline, we generate 100, 1,000, or 10,000 Bernoulli samples and select the single best sample (by the merit function) to warm-start Gurobi. Purely random sampling without solver refinement consistently fails to produce feasible solutions on the tightly-constrained IS and SC benchmarks, so we only report the Gurobi-refined results. The table below reports the absolute objective on all three benchmarks; PUMA +Gurobi dominates every naïve warm-start across all benchmarks, confirming that the advantage is not specific to a single problem family.

